说明：本软件完整源程序经整理共270页。现根据软件功能主次排序后，按照软件著作权登记源程序鉴别材料要求，提交连续前30页和连续后30页，共60页。每页不少于50行。

```
// ===== 文件：lib/features/game_2048/providers/game_provider.dart =====
0001  import 'dart:async';
0002  import 'dart:convert';
0003  import 'dart:math';
0004  import 'package:flutter/widgets.dart';
0005  import 'package:yueyou/core/database/storage_service.dart';
0006  import 'package:yueyou/core/utils/cyber_logger.dart';
0007  import 'package:yueyou/features/audio/services/sfx_service.dart';
0008  import 'package:yueyou/features/game_2048/domain/tile_model.dart';
0009  import 'package:flutter_riverpod/flutter_riverpod.dart';
0010  import 'package:yueyou/features/settings/providers/settings_provider.dart';
0011  import 'package:yueyou/features/audio/services/tts_engine_service.dart';
0013  final gameProvider = ChangeNotifierProvider<GameProvider>((ref) {
0014    final gp = GameProvider();
0015    gp.soundEnabled = ref.read(settingsProvider).sound;
0016    ref.listen(settingsProvider, (_, next) {
0017      gp.soundEnabled = next.sound;
0018    });
0019    gp.onUserMove = () => ref.read(ttsEngineProvider).notifyUserActivity();
0020    return gp;
0021  });
0023  /// 移动方向枚举 (映射 JS L149-152)
0024  enum Direction { up, down, left, right }
0026  /// 2048 游戏逻辑的核心 Provider (ChangeNotifier)
0027  /// 溯源：完整复刻自旧版 yueyou-app/www/js/modules/GameEngine.js
0028  class GameProvider extends ChangeNotifier with WidgetsBindingObserver {
0029    // 棋盘大小 (溯源：JS L16)
0030    final int size = 4;
0032    // 4x4 棋盘 (溯源：JS L28-31)
0033    List<List<TileModel?>> _board = [];
0034    List<List<TileModel?>> get board => _board;
0036    // 游戏实时分 (溯源：JS L17)
0037    int _score = 0;
0038    int get score => _score;
0040    // 最佳得分 (溯源：JS L10)
0041    int _bestScore = 0;
0042    int get bestScore => _bestScore;
0044    // 实时 Combo (溯源：JS L18)
0045    int _combo = 0;
0046    int get combo => _combo;
0048    // 最大 Combo (溯源：JS L11)
0049    int _maxCombo = 0;
0050    int get maxCombo => _maxCombo;
0052    // 下一个产生的方块 ID (溯源：JS L19)
0053    int _nextId = DateTime.now().millisecondsSinceEpoch;
0055    // 游戏是否结束 (溯源：JS L20)
0056    bool _isOver = false;
0057    bool get isOver => _isOver;
0059    // 随机数生成器 (替换 JS Math.random，支持注入种子用于测试)
```

```
0060    late final Random _random;
0062    /// 是否开启音效（由 SettingsProvider 通过 main.dart 同步注入）
0063    bool _soundEnabled = true;
0064    bool get soundEnabled => _soundEnabled;
0065    set soundEnabled(bool value) {
0066      if (_soundEnabled != value) {
0067        _soundEnabled = value;
0068        notifyListeners();
0069      }
0070    }
0072    /// 上一次滑动方向（供吉祥物眼球跟随使用）
0073    Direction? _lastMoveDirection;
0074    Direction? get lastMoveDirection => _lastMoveDirection;
0076    /// 用户有效操作回调（由 main.dart 注入，用于重置 TTS 空闲计时器）
0077    void Function()? onUserMove;
0079    /// 本次移动中合并产生的最大值（0=无合并，供吉祥物欢呼判断）
0080    int _lastMergedValue = 0;
0081    int get lastMergedValue => _lastMergedValue;
0083    /// 本次有有效滑动但无任何合并（供吉祥物惋惜/生气表情）
0084    bool _lastMoveNoMerge = false;
0085    bool get lastMoveNoMerge => _lastMoveNoMerge;
0087    /// 音效服务（支持注入 mock，默认使用 SfxService）
0088    final void Function(int)? _onPlayMerge;
0089    Timer? _persistDebounceTimer;
0090    final Duration _persistDebounceDuration;
0091    final StreamController<void> _gameOverController =
0092        StreamController<void>.broadcast();
0094    Stream<void> get onGameOver => _gameOverController.stream;
0096    GameProvider({
0097      Random? random,
0098      void Function(int)? onPlayMerge,
0099      bool autoLoadState = true,
0100      Duration persistDebounceDuration = const Duration(seconds: 1),
0101    })  : _onPlayMerge = onPlayMerge,
0102          _persistDebounceDuration = persistDebounceDuration {
0103      _random = random ?? Random();
0104      WidgetsBinding.instance.addObserver(this);
0105      if (autoLoadState) {
0106        _loadSavedState();
0107      } else {
0108        _initFresh();
0109      }
0110    }
0112    /// App 启动时从 StorageService 恢复游戏快照（对应 JS loadSavedState）
0113    void _loadSavedState() {
0114      _bestScore = StorageService.loadBestScore();
0115      _maxCombo = StorageService.loadMaxCombo();
0116      final saved = StorageService.loadGameState();
0117      if (saved != null) {
0118        try {
```

```
        final rawValue = saved['board_data'];
        final boardRaw = rawValue is String ? rawValue : null;
        if (boardRaw != null) {
          {
            final List<dynamic> rows = List<dynamic>.from(
              (boardRaw.isNotEmpty ? jsonDecode(boardRaw) : null) ?? [],
            );
            if (rows.length == size) {
              _board = List.generate(size, (r) {
                final row = rows[r] as List<dynamic>;
                return List.generate(size, (c) {
                  final cell = row[c];
                  if (cell == null) return null;
                  final m = cell as Map<String, dynamic>;
                  return TileModel(
                    id: (m['id'] as num).toInt(),
                    value: (m['value'] as num).toInt(),
                  );
                });
              });
              _score = (saved['score'] as num?)?.toInt() ?? 0;
              _combo = (saved['combo'] as num?)?.toInt() ?? 0;
              _bestScore = (saved['bestScore'] as num?)?.toInt() ?? _bestScore;
              _maxCombo = (saved['maxCombo'] as num?)?.toInt() ?? _maxCombo;
              int maxId = 0;
              for (final row in _board) {
                for (final tile in row) {
                  if (tile != null && tile.id > maxId) maxId = tile.id;
                }
              }
              _nextId = maxId + 1;
              notifyListeners();
              return;
            }
          }
        }
      } catch (e, stack) {
        CyberLogger.captureWarning(
          e,
          stack: stack,
          tag: 'game',
          extra: {'context': '游戏快照恢复失败，新开一局'},
        );
      }
    }
    // 无存档或解析失败则新开一局
    _initFresh();
  }
  void _initFresh() {
    _isOver = false;
```

```
    _score = 0;
    _combo = 0;
    _board = List.generate(size, (_) => List.filled(size, null));
    addRandomTile();
    addRandomTile();
    updateScore();
    notifyListeners();
  }
  @visibleForTesting
  void setStateForTesting({
    List<List<TileModel?>>? board,
    int? score,
    int? combo,
    bool? isOver,
  }) {
    if (board != null) _board = board;
    if (score != null) _score = score;
    if (combo != null) _combo = combo;
    if (isOver != null) _isOver = isOver;
    notifyListeners();
  }
  /// 初始化/重置游戏
  /// 溯源：映射 JS L15-26 (reset)
  void reset() {
    _isOver = false;
    _score = 0;
    _combo = 0;
    _board = List.generate(size, (_) => List.filled(size, null));
    addRandomTile();
    addRandomTile();
    updateScore();
    _schedulePersistState();
    notifyListeners();
  }
  /// 执行移动逻辑 (核心迁移逻辑)
  /// 溯源：映射 JS L32-112 (move)
  void move(Direction direction) {
    onUserMove?.call();
    _lastMoveDirection = direction;
    _lastMergedValue = 0;
    _lastMoveNoMerge = false;
    if (_isOver) {
      _emitGameOver();
      return;
    }
    bool moved = false;
    // 记录是否有任何合并发生，用于维护 combo
    final List<Map<String, dynamic>> mergedTiles = [];
    // P1-4：滑动音效/触觉反馈必须在确认实际位移后才能触发，
    // 否则在棋盘已满或边角无效滑动时会出现"按一下就响一下"的体感杂音。
```

```
    // 故移除原本 move() 入口处的预触发块，改在下方 `if (moved)` 内统一处理。
    // 获取移动向量 (溯源：JS L147-154)
    final vector = _getVector(direction);
    // 获取遍历顺序 (溯源：JS L155-161)
    final traversal = _getTraversalOrder(vector);
    // 记录本轮是否已经合并过的标记矩阵 (溯源：JS L46-48)
    final List<List<bool>> mergedFlags =
        List.generate(size, (_) => List.filled(size, false));
    // 开始遍历
    for (int r in traversal['rows']!) {
      for (int c in traversal['cols']!) {
        final TileModel? tile = _board[r][c];
        if (tile == null) continue;
        // 查找最远的可移动位置 (溯源：JS L54-60)
        int curR = r;
        int curC = c;
        int nextR = r + vector['y']!;
        int nextC = c + vector['x']!;
        // 循环推移 (溯源：JS L56-62)
        while (_inBounds(nextR, nextC) && _board[nextR][nextC] == null) {
          _board[nextR][nextC] = tile;
          _board[curR][curC] = null;
          curR = nextR;
          curC = nextC;
          nextR = curR + vector['y']!;
          nextC = curC + vector['x']!;
          moved = true; // 发生了位移
        }
        // 检查合并 (溯源：JS L63-82)
        if (_inBounds(nextR, nextC)) {
          final TileModel? targetTile = _board[nextR][nextC];
          // 逻辑：值相等且目标位置未在本轮合并过 (JS L65)
          if (targetTile != null &&
              targetTile.value == tile.value &&
              !mergedFlags[nextR][nextC]) {
            // 合并操作：值 * 2 (溯源：JS L66)
            _board[nextR][nextC] =
                targetTile.copyWith(value: targetTile.value * 2);
            _board[curR][curC] = null;
            // 标记已合并，并更新状态
            mergedFlags[nextR][nextC] = true;
            moved = true;
            _combo++; // (溯源：JS L70)
            // 更新最大 Combo (溯源：JS L71-74)
            if (_combo > _maxCombo) {
              _maxCombo = _combo;
            }
            // 存入合并列表（在 JS 里用于触发 3D 效果，此处预留）
            mergedTiles.add(
              {'r': nextR, 'c': nextC, 'value': _board[nextR][nextC]!.value},
```

```
            );
          }
        }
      }
    }
    if (moved) {
      // 没有任何合并时 combo 归零（溯源：JS L87）
      if (mergedTiles.isEmpty) {
        _combo = 0;
        _lastMoveNoMerge = true; // 有效滑动但无合并
      } else {
        // 记录本次合并的最大值，供吉祥物欢呼动画使用
        _lastMergedValue = mergedTiles
            .map((e) => e['value'] as int)
            .reduce((a, b) => a > b ? a : b);
      }
      // 添加新的随机块（溯源：JS L89）
      addRandomTile();
      // 更新全盘总分（溯源：JS L90）
      updateScore();
      // 判断是否无路可走（溯源：JS L91）
      if (!_movesAvailable()) {
        _markGameOver();
      }
      // 合并音效（对应 JS: if (result.mergedTiles.length > 0 && t.sound) l.playEffect('merge')）
      if (mergedTiles.isNotEmpty && _soundEnabled) {
        if (_onPlayMerge != null) {
          _onPlayMerge(_lastMergedValue);
        } else {
          SfxService.playMerge(_lastMergedValue);
        }
      } else if (_soundEnabled) {
        // P1-4：有效滑动但无合并 → 仅给一次轻量级触觉反馈，
        // 避免无效滑动空响。playMerge 内部已自带 playMoveFeedback，
        // 所以仅在无合并分支补一次。
        // 取盘面最大值作为震动强度参考，与旧实现的"分级震动"语义保持一致。
        int maxBoardValue = 0;
        for (final row in _board) {
          for (final tile in row) {
            if (tile != null && tile.value > maxBoardValue) {
              maxBoardValue = tile.value;
            }
          }
        }
        if (_onPlayMerge != null) {
          _onPlayMerge(maxBoardValue);
        } else {
          SfxService.playMoveFeedback(maxBoardValue);
        }
      }
```

```
      _schedulePersistState();
      notifyListeners();
      return;
    }
    // 即使没有发生位移，也可能已经进入无路可走状态
    if (!_movesAvailable()) {
      _markGameOver();
      _schedulePersistState();
      notifyListeners();
    }
  }
  void _markGameOver() {
    _isOver = true;
    _emitGameOver();
  }
  void _emitGameOver() {
    if (!_gameOverController.isClosed) {
      _gameOverController.add(null);
    }
  }
  /// 全盘得分计算逻辑
  ///  指令：严禁使用标准 2048 的“累加得分法”，必须严格遵循旧版的“全盘求和法”
  /// 溯源：映射 JS L113-119 (updateScore)
  void updateScore() {
    int total = 0;
    for (int r = 0; r < size; r++) {
      for (int c = 0; c < size; c++) {
        if (_board[r][c] != null) {
          total += _board[r][c]!.value;
        }
      }
    }
    _score = total;
    // 记录最佳分数 (溯源：JS L130-132)
    if (_score > _bestScore) {
      _bestScore = _score;
    }
  }
  /// 持久化当前快照到 StorageService（对应 JS saveLocalState）
  void _persistState() {
    final boardJson = List.generate(
      size,
      (r) => List.generate(size, (c) {
        final t = _board[r][c];
        if (t == null) return null;
        return <String, dynamic>{'id': t.id, 'value': t.value};
      }),
    );
    final int novelIndex = StorageService.getCurrentNovelIndex();
    final String? currentNovelId = StorageService.getCurrentNovelId();
```

```
    StorageService.saveGameState(
      board: boardJson,
      score: _score,
      combo: _combo,
      bestScore: _bestScore,
      maxCombo: _maxCombo,
      novelIndex: novelIndex,
      currentNovelId: currentNovelId,
    );
  }
  void _schedulePersistState() {
    _persistDebounceTimer?.cancel();
    if (_persistDebounceDuration <= Duration.zero) {
      _persistState();
      return;
    }
    _persistDebounceTimer = Timer(_persistDebounceDuration, _persistState);
  }
  void flushPersistState() {
    _persistDebounceTimer?.cancel();
    _persistDebounceTimer = null;
    _persistState();
  }
  @override
  void didChangeAppLifecycleState(AppLifecycleState state) {
    if (state == AppLifecycleState.paused) {
      flushPersistState();
    }
  }
  /// 在空格处添加随机块 (2 或 4)
  /// 溯源：映射 JS L179-191 (addRandomTile)
  void addRandomTile() {
    final List<Map<String, int>> emptyCells = [];
    for (int r = 0; r < size; r++) {
      for (int c = 0; c < size; c++) {
        if (_board[r][c] == null) {
          emptyCells.add({'r': r, 'c': c});
        }
      }
    }
    if (emptyCells.isNotEmpty) {
      final cell = emptyCells[_random.nextInt(emptyCells.length)];
      // 生成概率：2 (90%), 4 (10%) (溯源：JS L188)
      final int value = _random.nextDouble() < 0.9 ? 2 : 4;
      _board[cell['r']!][cell['c']!] = TileModel(id: _nextId++, value: value);
    }
  }
  /// 检查边界 (溯源：JS L144-146)
  bool _inBounds(int r, int c) {
    return r >= 0 && r < size && c >= 0 && c < size;
```

```
  }
  /// 移动向量映射（溯源：JS L147-154）
  Map<String, int> _getVector(Direction dir) {
    switch (dir) {
      case Direction.up:
        return {'x': 0, 'y': -1};
      case Direction.down:
        return {'x': 0, 'y': 1};
      case Direction.left:
        return {'x': -1, 'y': 0};
      case Direction.right:
        return {'x': 1, 'y': 0};
    }
  }
  /// 遍历顺序映射（溯源：JS L155-161）
  Map<String, List<int>> _getTraversalOrder(Map<String, int> vector) {
    List<int> rows = List.generate(size, (i) => i);
    List<int> cols = List.generate(size, (i) => i);
    // 如果是向下，则行序反转（JS L158）
    if (vector['y'] == 1) rows = rows.reversed.toList();
    // 如果是向右，则列序反转（JS L159）
    if (vector['x'] == 1) cols = cols.reversed.toList();
    return {'rows': rows, 'cols': cols};
  }
  /// 判定是否还有可行的移动（溯源：JS L162-178）
  bool _movesAvailable() {
    for (int r = 0; r < size; r++) {
      for (int c = 0; c < size; c++) {
        // 如果有空位，可用
        if (board[r][c] == null) return true;
        // 检查四个方向（溯源：JS L166-175）
        for (var dir in Direction.values) {
          final vector = _getVector(dir);
          final nextR = r + vector['y']!;
          final nextC = c + vector['x']!;
          if (_inBounds(nextR, nextC)) {
            final target = board[nextR][nextC];
            // 如果目标是空位或值相等，则仍有移动空间
            if (target == null || target.value == board[r][c]!.value) {
              return true;
            }
          }
        }
      }
    }
    return false;
  }
  /// 彩蛋机制：通过黑客手段强行抹除指定 ID 的数据块
  void eliminateTileById(int id) {
    bool removed = false;
```

```
0509      for (int r = 0; r < size; r++) {
0510        for (int c = 0; c < size; c++) {
0511          if (_board[r][c]?.id == id) {
0512            _board[r][c] = null;
0513            removed = true;
0514            break;
0515          }
0516        }
0517        if (removed) break;
0518      }
0520      if (removed) {
0521        // P1-2：消除一个 tile 后必须刷新全盘得分。
0522        // 旧版"全盘求和"得分模式下，移除任意 tile 都会让总和减少，
0523        // 不调 updateScore() 会让 UI 显示一个虚高的旧分数（与盘面不一致）。
0524        // 同时若之前已 GameOver，复活后必须重判：依旧无路可走时维持 over，
0525        // 而不是只在 _movesAvailable()==true 时静默改写 _isOver。
0526        updateScore();
0527        if (_soundEnabled) {
0528          if (_onPlayMerge != null) {
0529            _onPlayMerge(2048);
0530          } else {
0531            SfxService.playMoveFeedback(2048);
0532          }
0533        }
0534        _isOver = !_movesAvailable();
0535        _schedulePersistState();
0536        notifyListeners();
0537      }
0538    }
0540    @override
0541    void dispose() {
0542      _persistDebounceTimer?.cancel();
0543      _gameOverController.close();
0544      WidgetsBinding.instance.removeObserver(this);
0545      super.dispose();
0546    }
0547  }
// ===== 文件结束：lib/features/game_2048/providers/game_provider.dart =====

// ===== 文件：lib/features/audio/services/sfx_service.dart =====
0001  import 'dart:math';
0002  import 'dart:typed_data';
0004  import 'package:audioplayers/audioplayers.dart';
0005  import 'package:flutter/services.dart';
0007  import 'package:yueyou/core/utils/audio_utils.dart';
0008  import 'package:yueyou/core/utils/cyber_logger.dart';
0010  /// 物理音效引擎（V4 — 基于旧版 Web 端已验证音效移植）
0011  ///
0012  /// 旧版 Web 端核心：440→880Hz 上行正弦扫频 + 0.12 音量 + 指数衰减 + 300ms
0013  /// 本版在此基础上做 4 阶段递进，保持极简单层架构。
```

```
0014 ///
0015 /// 零杂音保证：上行扫频使用解析式 chirp 相位公式
0016 /// φ(t) = 2π(ft + (f-f)t/2T)，连续可微，零相位突变。
0017 class SfxService {
0018   static const bool _enabled = true;
0019   static AudioPlayer? _mergePlayer;
0021   /// 预生成的 4 阶段音效 WAV
0022   static final List<Uint8List> _tierWavs = [];
0024   static Future<void> init() async {
0025     _mergePlayer = AudioPlayer();
0026     await _mergePlayer!.setAudioContext(
0027       AudioContext(
0028         android: const AudioContextAndroid(
0029           audioFocus: AndroidAudioFocus.none,
0030           contentType: AndroidContentType.sonification,
0031           usageType: AndroidUsageType.assistanceSonification,
0032         ),
0033       ),
0034     );
0035     await _mergePlayer!.setReleaseMode(ReleaseMode.stop);
0036     _tierWavs.clear();
0038     //  阶段一 (≤16)：轻盈上行——与旧版完全一致
0039     // 旧版原始参数：440→880Hz, 100ms 扫频, 0.12 音量, 300ms 时长
0040     _tierWavs.add(
0041       _generateMergeTone(
0042         freqStart: 440,
0043         freqEnd: 880,
0044         chirpMs: 100,
0045         decayRate: 16,
0046         durationMs: 300,
0047         volume: 0.12,
0048       ),
0049     );
0051     //  阶段二 (≤128)：稍宽音域 + 略厚
0052     _tierWavs.add(
0053       _generateMergeTone(
0054         freqStart: 400,
0055         freqEnd: 900,
0056         chirpMs: 110,
0057         decayRate: 14,
0058         durationMs: 340,
0059         volume: 0.15,
0060       ),
0061     );
0063     //  阶段三 (≤1024)：更深沉的上行
0064     _tierWavs.add(
0065       _generateMergeTone(
0066         freqStart: 350,
0067         freqEnd: 950,
0068         chirpMs: 120,
```

```
        decayRate: 12,
        durationMs: 380,
        volume: 0.18,
      ),
    );
    //  阶段四 (>1024)：宽幅上行 + 最饱满
    _tierWavs.add(
      _generateMergeTone(
        freqStart: 330,
        freqEnd: 1000,
        chirpMs: 130,
        decayRate: 10,
        durationMs: 420,
        volume: 0.22,
      ),
    );
  }
  static Future<void> playMoveFeedback(int mergedValue) async {
    if (!_enabled) return;
    if (mergedValue <= 16) {
      await HapticFeedback.lightImpact();
    } else if (mergedValue <= 128) {
      await HapticFeedback.mediumImpact();
    } else if (mergedValue <= 1024) {
      await HapticFeedback.heavyImpact();
    } else {
      await HapticFeedback.heavyImpact();
      await Future.delayed(const Duration(milliseconds: 40));
      await HapticFeedback.heavyImpact();
    }
  }
  /// 触发合并音效
  static Future<void> playMerge(int mergedValue) async {
    if (!_enabled) return;
    // 异步触发震动，不阻塞音频
    playMoveFeedback(mergedValue);
    if (_mergePlayer != null && _tierWavs.isNotEmpty) {
      final tier = _getTier(mergedValue);
      _mergePlayer!.stop().then((_) {
        return _mergePlayer!.play(BytesSource(_tierWavs[tier]));
      }).catchError((Object e, StackTrace stack) {
        CyberLogger.captureWarning(
          e,
          stack: stack,
          tag: 'audio',
          extra: {'context': '播放 2048 合并音效'},
        );
      });
    }
  }
```

```
0126  /// 4 阶段档位映射
0127  static int _getTier(int mergedValue) {
0128    if (mergedValue <= 16) return 0;
0129    if (mergedValue <= 128) return 1;
0130    if (mergedValue <= 1024) return 2;
0131    return 3;
0132  }
0134  /// 生成上行扫频合并音效 WAV（移植自旧版 Web 端）
0135  ///
0136  /// 旧版 Web Audio API 等效逻辑（JavaScript）：
0137  ///   振荡器频率设为起始频率 f0
0138  ///   指数渐变至终止频率 f1（chirpT 时间内）
0139  ///   增益从 vol 指数衰减至 0.001（duration 时间内）
0140  ///
0141  /// 本版用解析式 chirp 精确复现，零杂音：
0142  ///   φ(t) = 2π(ft + (f-f)t/2T)  [t < chirpT]
0143  ///   φ(t) = φ(chirpT) + 2πf(t-chirpT) [t ≥ chirpT]
0144  static Uint8List _generateMergeTone({
0145    required double freqStart,
0146    required double freqEnd,
0147    required int chirpMs,
0148    required double decayRate,
0149    required int durationMs,
0150    required double volume,
0151    int sampleRate = 44100,
0152  }) {
0153    final numSamples = (sampleRate * durationMs / 1000).round();
0154    final dataSize = numSamples * 2;
0155    final buffer = ByteData(44 + dataSize);
0156    AudioUtils.writeWavHeader(buffer, sampleRate, dataSize);
0158    final chirpT = chirpMs / 1000.0;
0159    final attackEnd = (sampleRate * 0.003).round();
0161    // 预计算 chirp 过渡点相位（保证连续）
0162    final phaseAtChirpEnd =
0163        2 * pi * (freqStart * chirpT + (freqEnd - freqStart) * chirpT / 2);
0165    for (int i = 0; i < numSamples; i++) {
0166      final t = i / sampleRate;
0168      // ---- 3ms 柔攻击 ----
0169      double attack = 1.0;
0170      if (i < attackEnd) {
0171        attack = 0.5 - 0.5 * cos(pi * i / attackEnd);
0172      }
0174      // ---- 解析式上行 chirp ----
0175      double phase;
0176      if (t < chirpT) {
0177        phase = 2 *
0178            pi *
0179            (freqStart * t + (freqEnd - freqStart) * t * t / (2 * chirpT));
0180      } else {
0181        phase = phaseAtChirpEnd + 2 * pi * freqEnd * (t - chirpT);
```

```
0182        }
0184        // ---- 指数衰减包络（复现 exponentialRampToValueAtTime）----
0185        final envelope = exp(-t * decayRate);
0187        final signal = sin(phase) * envelope * volume * attack;
0188        final sample = (signal * 32767).round().clamp(-32768, 32767);
0189        buffer.setInt16(44 + i * 2, sample, Endian.little);
0190      }
0191      return buffer.buffer.asUint8List();
0192    }
0194    static void dispose() {
0195      try {
0196        _mergePlayer?.dispose();
0197      } catch (_) {
0198        // 忽略平台通道异常（测试环境或热重载时可能发生）
0199      }
0200      _mergePlayer = null;
0201      _tierWavs.clear();
0202    }
0203  }
// ===== 文件结束：lib/features/audio/services/sfx_service.dart =====

// ===== 文件：lib/features/audio/services/tts_engine_service.dart =====
0001  import 'dart:async';
0002  import 'dart:io';
0004  import 'package:audioplayers/audioplayers.dart';
0005  import 'package:flutter/material.dart';
0006  import 'package:flutter_riverpod/flutter_riverpod.dart';
0008  import 'package:yueyou/core/config/tts_config.dart';
0009  import 'package:yueyou/core/constants/cyber_error_messages.dart';
0010  import 'package:yueyou/core/utils/cyber_logger.dart';
0011  import 'package:yueyou/core/utils/tts_cache_manager.dart';
0012  import 'package:yueyou/features/audio/domain/tts_audio_buffer.dart';
0013  import 'package:yueyou/features/audio/domain/tts_audio_models.dart';
0014  import 'package:yueyou/features/audio/domain/tts_engine_interfaces.dart';
0015  import 'package:yueyou/features/audio/domain/tts_http_models.dart';
0016  import 'package:yueyou/features/audio/domain/tts_network_interfaces.dart';
0017  import 'package:yueyou/features/audio/services/tts_audio_adapters.dart';
0018  import 'package:yueyou/features/audio/services/tts_audio_downloader.dart';
0019  import 'package:yueyou/features/audio/services/tts_audio_janitor.dart';
0020  import 'package:yueyou/features/audio/services/tts_diagnostics_service.dart';
0021  import 'package:yueyou/features/audio/services/tts_http_client.dart';
0022  import 'package:yueyou/features/settings/providers/settings_provider.dart';
0024  export 'package:yueyou/features/audio/domain/tts_audio_models.dart';
0025  export 'package:yueyou/features/audio/domain/tts_audio_buffer.dart'
0026      show TtsBufferStatus;
0027  export 'package:yueyou/features/audio/domain/tts_http_models.dart';
0028  export 'package:yueyou/features/audio/domain/tts_engine_interfaces.dart';
0029  export 'package:yueyou/features/audio/domain/tts_network_interfaces.dart';
0031  /// TTS 引擎全局 Provider（Riverpod 生命周期托管版）
0032  ///
```

```
0033 /// - 通过 [ref.watch] 声明式依赖 [settingsProvider]，实现配置热更新
0034 /// - 通过 [ref.onDispose] 自动回收引擎资源，消除手动 dispose 调用
0035 /// - 通过 [ref.listen] 响应设置变更，替代手动 addListener/removeListener
0036 final ttsEngineProvider = ChangeNotifierProvider<TtsEngineService>((ref) {
0037   final settings = ref.read(settingsProvider);
0038   final svc = TtsEngineService(settings, externalSettingsListener: false);
0039   // 注册 Riverpod 自动销毁钩子，消除双调用风险（幂等守卫见 dispose()）
0040   ref.onDispose(svc.dispose);
0041   // 监听设置变更，推送给引擎同步（替代 addListener 手动管理）
0042   ref.listen<SettingsProvider>(
0043     settingsProvider,
0044     (_, next) => svc.syncSettingsFromProvider(next),
0045   );
0046   return svc;
0047 });
0049 /// 阅游 TTS 音频执行层
0050 /// 架构：纯执行服务，由 TtsAudioNotifier 编排调度
0051 class TtsEngineService extends ChangeNotifier {
0052   final TtsConfig _config;
0054   // 核心状态
0055   bool _disposed = false;
0056   late final Future<void> Function(Duration) _delayFn;
0057   final Completer<void> _disposeCompleter = Completer<void>();
0059   String _voice = '';
0060   late double _volume;
0061   TtsPlaybackState _state = TtsPlaybackState.disabled;
0062   String? _lastError;
0063   int _errorTimestamp = 0;
0064   double _playbackRate = 1.0;
0065   final List<double> _speedTiers = [1.0, 1.2, 1.5, 2.0, 2.5, 0.7];
0067   // 执行层核心（通过接口抽象，支持测试注入）
0068   late final TtsAudioPlayer _audioPlayer;
0069   late final TtsWakeLock _wakeLock;
0070   late final TtsHttpClient _httpClient;
0071   late final TtsFallbackEngine _fallbackEngine;
0073   // 子系统（PR-C 抽出）：诊断 / 下载 由构造器注入依赖 callback 后构建，
0074   // 所有「写回引擎」的副作用都通过显式 callback 暴露，便于单元测试。
0075   late final TtsDiagnosticsService _diagnostics;
0076   late final TtsAudioDownloader _downloader;
0077   String? _fallbackNotification;
0078   StreamSubscription<Duration>? _positionSub;
0079   StreamSubscription<Duration>? _durationSub;
0080   int _loopSession = 0;
0081   bool _wakeLockHeld = false;
0083   // 影子字段（由 TtsAudioNotifier 通过 syncShadow 驱动，保持 ReaderProvider 兼容）
0084   TtsAudioItem? _currentItem;
0086   /// 当前播放任务的完成信号，用于外部强制停止（如切换发声人）
0087   Completer<void>? _playCompleter;
0089   /// 实时播放进度流 (0.0 -> 1.0)
0090   Stream<double> get progressStream => _progressController.stream;
```

```
0091  final _progressController = StreamController<double>.broadcast();
0092  Duration _currentDuration = Duration.zero;
0094  /// 硬件初始化 Future，用于守卫异步初始化流程
0095  late final Future<void> _initFuture;
0096  bool _initCompleted = false;
0098  /// 引擎硬件初始化完成 Future（测试 / 集成测试可 await 等待 _volume / _playbackRate
0099  /// 等 late 字段就绪，避免在初始化未完成前调用 syncSettingsFromProvider 触发
0100  /// LateInitializationError）。生产环境通常无需显式等待，硬件初始化是 fire-and-forget。
0101  Future<void> get initialized => _initFuture;
0102  String? _lastGeneratedAudioPath;
0104  TtsPlaybackState get state => _state;
0106  /// 引擎是否处于「已启用」状态
0107  bool get isEnabled => _state != TtsPlaybackState.disabled;
0109  /// 是否正在播放
0110  bool get isPlaying => _state == TtsPlaybackState.playing;
0112  /// 是否已暂停
0113  bool get isPaused => _state == TtsPlaybackState.paused;
0115  /// 是否处于错误状态
0116  bool get isError => _state == TtsPlaybackState.error;
0117  String? get lastError => _lastError;
0118  int get errorTimestamp => _errorTimestamp;
0119  double get playbackRate => _playbackRate;
0120  int get currentSession => _loopSession;
0121  TtsAudioItem? get currentItem => _currentItem;
0123  /// 缓冲队列上限（来自 TtsConfig.maxPrefetchQueue，默认 6）。
0124  int get maxBufferedCount => _config.maxPrefetchQueue;
0126  late SettingsProvider _settings;
0128  TtsEngineService(
0129    SettingsProvider settings, {
0130    TtsConfig? config,
0131    TtsAudioPlayer? audioPlayer,
0132    TtsWakeLock? wakeLock,
0133    TtsHttpClient? httpClient,
0134    Future<void> Function(Duration)? delayFn,
0135    TtsFallbackEngine? fallbackEngine,
0137    /// [externalSettingsListener]：是否由外部（Riverpod ref.listen）接管设置监听。
0138    /// - true（默认）：TtsEngineService 内部自行 addListener 监听 settings 变更（非 Riverpod 场景）。
0139    /// - false：由 ttsEngineProvider 的 ref.listen 统一推送，避免双重注册。
0140    bool externalSettingsListener = true,
0141  })  : _config = config ?? TtsConfig.current,
0142        _audioPlayer = audioPlayer ?? RealAudioPlayer(AudioPlayer()),
0143        _wakeLock = wakeLock ?? RealWakeLock(),
0144        _httpClient = httpClient ?? RealTtsHttpClient(RealHttpClient()),
0145        _fallbackEngine = fallbackEngine ?? FlutterTtsFallbackEngine() {
0146    _delayFn = delayFn ?? (d) => Future<void>.delayed(d);
0147    _settings = settings;
0148    _voice = settings.voice;
0149    // 配置音频上下文：允许朗读与背景音共存（Ducking 策略）
0150    _audioPlayer.setAudioContext(
0151      AudioContext(
```

```
        android: const AudioContextAndroid(
          audioFocus: AndroidAudioFocus.gainTransient,
          contentType: AndroidContentType.speech,
          usageType: AndroidUsageType.assistanceAccessibility,
        ),
        iOS: AudioContextIOS(
          category: AVAudioSessionCategory.playback,
          options: const {
            AVAudioSessionOptions.mixWithOthers,
            AVAudioSessionOptions.duckOthers,
          },
        ),
      ),
    );
    // 绑定物理进度监听，实现提词器绝对同步
    _positionSub = _audioPlayer.onPositionChanged.listen((pos) {
      if (_currentDuration.inMilliseconds > 0) {
        final progress = pos.inMilliseconds / _currentDuration.inMilliseconds;
        _progressController.add(progress.clamp(0.0, 1.0));
      }
    });
    _durationSub = _audioPlayer.onDurationChanged.listen((dur) {
      _currentDuration = dur;
    });
    // 根据设置初始化状态，确保 isEnabled 同步正确
    if (_settings.storyTts) {
      _state = TtsPlaybackState.buffering;
    }
    _initFuture = _initTtsHardware();
    // 实例化子系统：诊断 / 下载。所有依赖与状态写回 callback 全部
    // 显式传递，便于 Mock 替换。
    _diagnostics = TtsDiagnosticsService(
      httpClient: _httpClient,
      config: _config,
      voiceGetter: () => _initCompleted ? _voice : _settings.voice,
      onError: _setLastError,
      onClearError: _clearLastError,
    );
    _downloader = TtsAudioDownloader(
      httpClient: _httpClient,
      fallbackEngine: _fallbackEngine,
      config: _config,
      voiceGetter: () => _initCompleted ? _voice : _settings.voice,
      initFuture: _initFuture,
      isDisposed: () => _disposed,
      delayFn: _delayFn,
      playbackRateGetter: () => _playbackRate,
      onError: _setLastError,
      onClearError: _clearLastError,
      onFallbackNotification: _setFallbackNotification,
```

```
      onPathGenerated: (path) => _lastGeneratedAudioPath = path,
      progressEmitter: _progressController.add,
    );
    // 非 Riverpod 场景（如测试直接构造），仍使用内部监听器
    if (externalSettingsListener) {
      _listenToSettings();
    }
  }
  /// 供 Riverpod ref.listen 调用：接收最新 SettingsProvider 实例并同步配置
  /// 替代手动 addListener，实现声明式依赖解耦。
  void syncSettingsFromProvider(SettingsProvider newSettings) {
    // 更新内部 settings 引用（Riverpod 可能重建 SettingsProvider）
    _settings = newSettings;
    _onSettingsChanged();
  }
  void _listenToSettings() {
    _settings.addListener(_onSettingsChanged);
  }
  void _onSettingsChanged() {
    if (_disposed) return;
    if (_settings.voice != _voice) {
      _voice = _settings.voice;
      refreshSession();
    }
    _syncSettingsInternal(
      storyTts: _settings.storyTts,
      ttsRate: _settings.ttsRate,
      voice: _settings.voice,
      volume: _settings.ambientVol,
    );
  }
  void _safeSetPlaybackRate(double rate) {
    unawaited(
      _audioPlayer.setPlaybackRate(rate).catchError((Object e) {
        _setLastError(CyberErrorMessages.ttsAudioParamFailed);
        CyberLogger.captureWarning(
          e,
          tag: 'tts',
          extra: {'context': '设置播放倍速失败'},
        );
      }),
    );
  }
  void _safeSetVolume(double volume) {
    unawaited(
      _audioPlayer.setVolume(volume).catchError((Object e) {
        _setLastError(CyberErrorMessages.ttsAudioParamFailed);
        CyberLogger.captureWarning(
          e,
          tag: 'tts',
```

```
          extra: {'context': '设置音量失败'},
        );
      }),
    );
  }
  void _setLastError(dynamic error) {
    final message = switch (error) {
      final String msg => msg,
      TimeoutException _ => '接入链路波动，请检查网络',
      SocketException _ => '网络连接失败，请检查网络设置',
      HttpException _ => '服务器连接异常，请稍后重试',
      FormatException _ => '数据解析失败，请联系管理员',
      final int code => switch (code) {
          404 => '请求的资源不存在，请检查后重试',
          >= 500 => '服务器维护中，请稍后再试',
          >= 400 => '请求参数异常，请稍后重试',
          _ => '网络请求异常 ($code)',
        },
      _ => '系统服务异常，请稍后重试',
    };
    _lastError = message;
    _errorTimestamp = DateTime.now().millisecondsSinceEpoch;
    notifyListeners();
  }
  void _clearLastError() {
    if (_lastError == null) return;
    _lastError = null;
    notifyListeners();
  }
  /// 清理最近一次 TTS 错误，供 UI 层在提示后主动清空。
  void clearLastError() {
    _clearLastError();
  }
  /// 设置 TTS 错误信息，供外部模块（如 ReaderProvider 前置拦截）使用。
  void setLastError(dynamic error) {
    _setLastError(error);
  }
  String? get fallbackNotification => _fallbackNotification;
  void _setFallbackNotification(String message) {
    _fallbackNotification = message;
    notifyListeners();
  }
  /// 清理降级通知，供 UI 层在显示 Toast 后主动清空。
  void clearFallbackNotification() {
    _fallbackNotification = null;
  }
  void _syncSettingsInternal({
    required bool storyTts,
    required double ttsRate,
    required String voice,
```

```
    required double volume,
  }) {
    if (_playbackRate != ttsRate) {
      _playbackRate = ttsRate;
      _safeSetPlaybackRate(ttsRate);
    }
    if (_volume != volume) {
      _volume = volume.clamp(0.0, 1.0);
      _safeSetVolume(_volume);
    }
    // 语音/设置变更仅同步硬件参数，状态迁移由 TtsAudioNotifier 负责。
  }
  @override
  void dispose() {
    // 幂等守卫：防止 Riverpod ref.onDispose + 外部手动 dispose 双调用
    if (_disposed) return;
    _disposed = true;
    if (!_disposeCompleter.isCompleted) _disposeCompleter.complete();
    _settings.removeListener(_onSettingsChanged);
    _positionSub?.cancel();
    _durationSub?.cancel();
    _progressController.close();
    TtsCacheManager.instance.stopPeriodicClean();
    _wakeLockHeld = false;
    unawaited(_wakeLock.disable());
    unawaited(_audioPlayer.dispose());
    unawaited(_fallbackEngine.dispose());
    unawaited(_downloader.deleteFileIfExists(_lastGeneratedAudioPath));
    _lastGeneratedAudioPath = null;
    unawaited(_initFuture); // 确保初始化完成（即使是销毁时）
    super.dispose();
  }
  Future<void> _initTtsHardware() async {
    try {
      await cleanupOrphanedTtsFiles(
        activePathGetter: () => _lastGeneratedAudioPath,
      );
      _voice = _settings.voice;
      _volume = _settings.ambientVol;
      _playbackRate = _settings.ttsRate;
      await _audioPlayer.setVolume(_volume.clamp(0.0, 1.0));
      await _audioPlayer.setPlaybackRate(_playbackRate);
      _initCompleted = true;
      try {
        await _fallbackEngine.initialize();
      } catch (e, st) {
        CyberLogger.captureWarning(
          e,
          stack: st,
          tag: 'tts',
```

```
          extra: {'context': '本地 TTS 降级引擎初始化失败'},
        );
      }
      CyberLogger.captureMessage('TTS 执行层已初始化', tag: 'tts');
    } catch (e, st) {
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'tts',
        extra: {'context': 'TTS 引擎初始化失败'},
      );
      _initCompleted = true;
    }
  }
  Future<void> init() async {
    // 启动缓存定期清理（30 分钟间隔，跳过当前播放文件）
    TtsCacheManager.instance.startPeriodicClean(
      excludeActivePath: () => _lastGeneratedAudioPath,
    );
  }
  /// 轻量 ping 探测：委托 [TtsDiagnosticsService] 检测服务端可达性。
  Future<bool> pingServer() => _diagnostics.pingServer();
  /// 手动触发 TTS 缓存清理（供设置页「清理缓存」按钮调用）
  ///
  /// 执行双重判定清理：超过 24 小时的文件 + 总大小超过 500MB 时按旧删。
  /// 跳过当前正在播放的文件，幂等安全。
  Future<TtsCacheCleanResult> cleanCacheNow() {
    return TtsCacheManager.instance.cleanNow(
      excludePath: _lastGeneratedAudioPath,
    );
  }
  /// 查询当前 TTS 缓存占用信息（不执行清理）
  Future<TtsCacheStat> getCacheStat() {
    return TtsCacheManager.instance.getStat();
  }
  void cycleSpeed() {
    final int currentIndex = _speedTiers.indexOf(_playbackRate);
    final int nextIndex = (currentIndex + 1) % _speedTiers.length;
    _playbackRate = _speedTiers[nextIndex];
    // P3 修复：统一走 `_safeSetPlaybackRate`（内部 `unawaited(.catchError)`），
    // 与 `_syncSettingsInternal` 的容错口径一致：播放器底层故障时必走
    // `_setLastError + captureWarning`，不向 UI 层抛出未处理异常导致红屏。
    _safeSetPlaybackRate(_playbackRate);
    _settings.setTtsRate(_playbackRate);
    notifyListeners();
  }
  void syncSpeedFromSettings(double logicalRate, double hardwareRate) {
    if (_playbackRate == logicalRate) return;
    _playbackRate = logicalRate;
    // P3 修复：同 `cycleSpeed`，改走 `_safeSetPlaybackRate` 统一容错。
```

```
    _safeSetPlaybackRate(logicalRate);
    notifyListeners();
  }
  // ——— 兼容字段与 getter（旧调用方/测试使用） ————————————————————————————————
  final List<String> _compatBuffer = [];
  /// @deprecated 缓冲管理已迁移至 TtsAudioNotifier + TtsAudioBuffer。
  int get bufferedCount => _compatBuffer.length;
  double get bufferHealthRatio {
    // 保护：如果队列上限为 0，视为健康度满分（1.0），防止除零
    if (_config.maxPrefetchQueue <= 0) return 1.0;
    return (bufferedCount / _config.maxPrefetchQueue).clamp(0.0, 1.0);
  }
  TtsBufferStatus get bufferStatus {
    if (bufferedCount == 0) return TtsBufferStatus.empty;
    final ratio = bufferHealthRatio;
    if (ratio >= 0.6) return TtsBufferStatus.healthy;
    if (ratio >= 0.33) return TtsBufferStatus.warning;
    return TtsBufferStatus.critical;
  }
  /// @deprecated 句子源管理已迁移至 TtsAudioNotifier.registerSentenceSource。
  Future<TtsAudioRequest?> Function(int session)? onNeedPrefetch;
  FutureOr<void> Function(TtsAudioItem item)? onItemStarted;
  FutureOr<void> Function(TtsAudioItem item)? onItemFinished;
  // ——— 兼容存根（旧调用方/测试使用，新代码走 TtsAudioNotifier） ——————————————
  /// 通知引擎有用户交互行为，触发监听者（TtsAudioNotifier）重置计时器。
  void notifyUserActivity() {
    notifyListeners();
  }
  /// @deprecated 使用 TtsAudioNotifier.setEnabled 替代。
  void setEnabled(bool enabled) {
    if (_disposed) return;
    if (enabled) {
      if (_state == TtsPlaybackState.disabled) {
        syncShadow(state: TtsPlaybackState.buffering);
      }
    } else {
      stopAll();
    }
  }
  /// @deprecated 使用 TtsAudioNotifier.refreshSession 替代。
  void refreshSession() {
    if (_disposed) return;
    _loopSession++;
    _audioPlayer.stop();
    _currentItem = null;
    syncShadow(state: TtsPlaybackState.disabled);
  }
  /// @deprecated 使用 TtsAudioNotifier.play 替代。
  Future<void> play() async {
    if (_disposed) return;
```

```
    if (_state == TtsPlaybackState.paused) {
      syncShadow(state: TtsPlaybackState.playing);
    } else if (_state == TtsPlaybackState.disabled) {
      syncShadow(state: TtsPlaybackState.buffering);
    }
  }
  /// @deprecated 使用 TtsAudioNotifier.pause 替代。
  Future<void> pause() async {
    if (_disposed) return;
    await _audioPlayer.pause();
    await _wakeLock.disable();
    syncShadow(state: TtsPlaybackState.paused);
  }
  /// 停止所有音频播放（由 TtsAudioNotifier 在切换会话时调用）。
  Future<void> stopAll() async {
    _loopSession++;
    await Future.wait([
      _audioPlayer.stop(),
      _fallbackEngine.stop(),
      _wakeLock.disable(),
    ]);
    _currentItem = null;
    syncShadow(state: TtsPlaybackState.disabled);
  }
  ///  TTS 连接测试工具：委托 [TtsDiagnosticsService]，返回详细的诊断信息。
  Future<Map<String, dynamic>> testConnection() =>
      _diagnostics.testConnection();
  Future<void> _syncWakeLock(bool enable) async {
    if (_wakeLockHeld == enable) return;
    _wakeLockHeld = enable;
    try {
      if (enable) {
        await _wakeLock.enable();
      } else {
        await _wakeLock.disable();
      }
    } catch (_) {}
  }
  /// 使用本地 TTS 引擎朗读指定文本：委托 [TtsAudioDownloader]。
  Future<bool> speakWithLocalTts(String text) =>
      _downloader.speakWithLocalTts(text);
  /// 下载 TTS 音频文件（带重试机制）：委托 [TtsAudioDownloader]，
  /// 返回文件路径或 null。
  Future<String?> downloadAudio(TtsAudioRequest request) =>
      _downloader.downloadAudio(request);
  /// 播放指定的本地音频文件。
  ///
  /// [path] 是本地临时文件路径，[onComplete] 在播放正常结束或出错时调用。
  Future<void> playFile(String path, {void Function()? onComplete}) async {
    try {
```

```
      // 物理进度归零，确保提词器扫光从头开始
      _currentDuration = Duration.zero;
      _progressController.add(0.0);
      // 先停止当前播放，确保播放器处于干净状态（避免 MEDIA_ERROR_SERVER_DIED）
      await _audioPlayer.stop();
      // 检查文件是否存在且有效
      final file = File(path);
      if (!await file.exists()) {
        onComplete?.call();
        return;
      }
      final fileSize = await file.length();
      if (fileSize < 1024) {
        CyberLogger.captureWarning(
          StateError('TTS 音频文件太小'),
          tag: 'tts',
          extra: {
            'context': '文件太小，跳过播放',
            'sizeBytes': '$fileSize',
            'path': path,
          },
        );
        onComplete?.call();
        return;
      }
      await _audioPlayer.setSource(DeviceFileSource(path)).timeout(
            const Duration(seconds: 5),
            onTimeout: () => throw TimeoutException('setSource 超时'),
          );
      if (_disposed) return;
      _playCompleter = Completer<void>();
      final sub = _audioPlayer.onPlayerComplete.listen((_) {
        if (_playCompleter?.isCompleted == false) _playCompleter?.complete();
      });
      try {
        await _audioPlayer.resume();
        // 等待播放自然完成，或被 stopAudio 强制完成
        await _playCompleter!.future.timeout(
          const Duration(seconds: 30),
          onTimeout: () {
            unawaited(_audioPlayer.stop());
          },
        );
        // P1-1：原代码在每句播完后调用 _syncWakeLock(false)，
        // 但下一句进入相同 TtsPlaybackState.playing 时 syncShadow 因 _state 未变化
        // 不会重新申请 wakelock，导致从第二句起屏幕熄灭。
        // 现在 wakelock 完全由状态机驱动：进入 playing/buffering 时申请，
        // 由 stopAll/pause/dispose 集中释放，单句结束不再触发释放。
        onComplete?.call();
      } finally {
```

```
        await sub.cancel();
        _playCompleter = null;
      }
    } on TimeoutException catch (e, st) {
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'tts',
        extra: {'context': 'playFile 超时，播放熔断'},
      );
      await _audioPlayer.stop();
      _setLastError(CyberErrorMessages.ttsAudioLoadTimeout);
      onComplete?.call();
    } catch (e, st) {
      CyberLogger.captureWarning(
        e is Exception ? e : Exception('$e'),
        stack: st,
        tag: 'tts',
        extra: {'context': 'playFile 异常'},
      );
      // 尝试停止播放器，出错时静默处理
      try {
        await _audioPlayer.stop();
      } catch (_) {}
      onComplete?.call();
    }
  }
  /// 恢复音频播放（已暂停时）。
  Future<void> resumeAudio() async => _audioPlayer.resume();
  /// 暂停音频播放。
  Future<void> pauseAudio() async {
    await Future.wait([
      _audioPlayer.pause(),
      _fallbackEngine.stop(),
    ]);
  }
  /// 停止所有音频播放。
  Future<void> stopAudio() async {
    // 1. 强制释放 playFile 的 await 阻塞
    if (_playCompleter?.isCompleted == false) {
      _playCompleter?.complete();
    }
    // 2. 底层硬件停播
    await Future.wait([
      _audioPlayer.stop(),
      _fallbackEngine.stop(),
    ]);
    _compatBuffer.clear();
  }
  /// 影子状态同步（由 [TtsAudioNotifier] 调用，保持向下兼容）。
```

```
0658    void syncShadow({
0659      TtsPlaybackState? state,
0660      int? session,
0661      String? error,
0662      TtsAudioItem? item,
0663      String? fallbackMessage,
0664    }) {
0665      bool changed = false;
0666      if (state != null && _state != state) {
0667        _state = state;
0668        _syncWakeLock(
0669          state == TtsPlaybackState.playing ||
0670              state == TtsPlaybackState.buffering,
0671        );
0672        changed = true;
0673      }
0674      if (session != null && _loopSession != session) {
0675        _loopSession = session;
0676      }
0677      if (error != null && _lastError != error) {
0678        _lastError = error;
0679        _errorTimestamp = DateTime.now().millisecondsSinceEpoch;
0680        changed = true;
0681      }
0682      if (error == null && _lastError != null) {
0683        _lastError = null;
0684        changed = true;
0685      }
0686      if (item != null || (item == null && _currentItem != null)) {
0687        _currentItem = item;
0688      }
0689      if (fallbackMessage != null) {
0690        _fallbackNotification = fallbackMessage;
0691        changed = true;
0692      } else if (fallbackMessage == null && _fallbackNotification != null) {
0693        _fallbackNotification = null;
0694      }
0695      if (changed && !_disposed) notifyListeners();
0696    }
0697  }
// ===== 文件结束：lib/features/audio/services/tts_engine_service.dart =====

// ===== 文件：lib/features/audio/providers/tts_audio_notifier.dart =====
0001  import 'dart:async';
0002  import 'dart:io';
0004  import 'package:flutter_riverpod/flutter_riverpod.dart';
0006  import 'package:yueyou/core/utils/cyber_logger.dart';
0007  import 'package:yueyou/features/audio/domain/tts_audio_buffer.dart';
0008  import 'package:yueyou/features/audio/domain/tts_audio_state.dart';
0009  import 'package:yueyou/features/audio/domain/tts_audio_state_helpers.dart';
```

```
import 'package:yueyou/features/audio/providers/tts_fallback_controller.dart';
import 'package:yueyou/features/audio/providers/tts_paused_interrupt_guard.dart';
import 'package:yueyou/features/audio/services/tts_engine_service.dart';
import 'package:yueyou/features/settings/providers/settings_provider.dart';
/// TTS 音频流状态机 Provider。
final ttsAudioProvider =
    NotifierProvider<TtsAudioNotifier, TtsAudioState>(TtsAudioNotifier.new);
/// TTS 音频流状态机编排层。
///
/// ## 架构
///
/// 双轨并行生产者/消费者模型：
/// - **预加载轨道**：`_prefetchRunner` 后台持续下载音频文件，填充 `TtsAudioBuffer`
/// - **播放轨道**：`_playRunner` 串行消费缓冲队列，播完即焚
/// - 两轨道通过共享 `TtsAudioBuffer` 通信，互不阻塞
///
/// ## 状态流转
///
/// Idle ——play()—— Buffering ——缓冲就绪—— Playing ——pause()—— Paused
///                  ▲                      |                    |
///                  |      缓冲耗尽          |    resumeAudio()   |
///                  └──────────────────────────────────────────┘ ──────────────────────┘
///                                          Error ——recover()—— Buffering
class TtsAudioNotifier extends Notifier<TtsAudioState> {
  late final TtsEngineService _engine;
  late final TtsAudioBuffer _buffer;
  TtsSentenceSource? _sentenceSource;
  TtsAudioItem? _currentItem;
  String? _currentFilePath; // 当前播放文件路径，用于暂停后恢复
  int _session = 0;
  double _playbackRate = 1.0;
  String? _fallbackMessage;
  bool _disposed = false;
  bool _pumpActive = false;
  int _consecutiveFailures = 0;
  bool _isPausing = false;
  bool _backgroundTolerant = false;
  bool _prefetchPaused = false;
  Timer? _idleTimer; // 静默暂停计时器
  // PR-D 抽出的子系统：暂停中断守卫 + 本地降级控制器。
  final TtsPausedInterruptGuard _pausedGuard = TtsPausedInterruptGuard();
  late final TtsFallbackController _fallback;
  /// 设置后台宽容模式：提高降级阈值，避免后台限网触发误降级。
  void setBackgroundTolerant(bool value) {
    _backgroundTolerant = value;
    if (value) {
      _prefetchPaused = true;
      _consecutiveFailures = 0;
    } else {
      _prefetchPaused = false;
```

```
      _consecutiveFailures = 0;
    }
  }
  /// 注册句子源（由 ReaderProvider 在构造时调用）。
  void registerSentenceSource(TtsSentenceSource source) {
    _sentenceSource = source;
  }
  /// TTS 是否处于活跃状态（播放或缓冲）。
  /// 供外部（如 ReaderProvider）在调用 refreshSession 前做守卫判断。
  /// 使用 Riverpod state（纯 Dart，每次 build 干净初始化），
  /// 比 native audioplayers 状态更可靠。
  bool get isActivelyPlaying {
    final s = state;
    return s is TtsAudioPlaying || s is TtsAudioBuffering;
  }
  @override
  TtsAudioState build() {
    _engine = ref.read(ttsEngineProvider);
    _buffer = TtsAudioBuffer(maxSize: _engine.maxBufferedCount);
    _playbackRate = _engine.playbackRate;
    _fallback = TtsFallbackController(
      engine: _engine,
      pausedGuard: _pausedGuard,
      sentenceSourceGetter: () => _sentenceSource,
      sessionGetter: () => _session,
      playbackRateGetter: () => _playbackRate,
      bufferGetter: () => _buffer,
      currentItemGetter: () => _currentItem,
      currentItemSetter: (item) => _currentItem = item,
      isDisposed: () => _disposed,
      applyState: _applyState,
      snapshotOf: snapshotOf,
      onConsecutiveFailuresReset: (n) => _consecutiveFailures = n,
      fallbackMessageSetter: (msg) => _fallbackMessage = msg,
    );
    ref.onDispose(() {
      _disposed = true;
      _pumpActive = false;
      _idleTimer?.cancel();
      _buffer.clear();
    });
    // 核心：监听引擎的心跳信号（notifyUserActivity 触发的 notifyListeners）
    ref.listen(ttsEngineProvider, (prev, next) {
      _resetIdleTimer();
    });
    // 核心：监听设置中的时长变更
    //
    // 修复 P1（dead code）：`settingsProvider` 是 ChangeNotifierProvider，
    // 在 notify 时 prev 与 next 引用同一个 SettingsProvider 实例，
    // 直接比较 `prev?.idleTimeout != next.idleTimeout` 永远 false，分支不会触发。
```

```
    // 改用 `select` 让 Riverpod 内部对 idleTimeout 数值做快照对比，
    // 只在数值真正变化时 fire callback。
    ref.listen<int>(
      settingsProvider.select((s) => s.idleTimeout),
      (prev, next) {
        if (prev != next) _resetIdleTimer();
      },
    );
    return TtsAudioIdle(playbackRate: _playbackRate, fallbackMessage: null);
  }
  /// 内部重置空闲计时器（静默暂停）。
  void _resetIdleTimer() {
    _idleTimer?.cancel();
    _idleTimer = null;
    final settings = ref.read(settingsProvider);
    final minutes = settings.idleTimeout;
    // 如果设置为 0（永不）或者当前未在播放/缓冲，则不启动计时
    if (minutes <= 0) return;
    if (state is TtsAudioIdle) return;
    _idleTimer = Timer(Duration(minutes: minutes), () {
      CyberLogger.captureMessage(
        '[TTS] 静默暂停：用户已空闲 ${minutes}m，自动停播',
        tag: 'tts',
      );
      pause();
    });
  }
  // ──── 公开 API ──────────────────────────────────────────────────────────────
  /// 启动或恢复播放。
  void play() {
    if (_disposed) return;
    _engine.clearLastError();
    if (_sentenceSource == null) {
      setBusinessError('无法开启 TTS：请先导入书籍');
      return;
    }
    _consecutiveFailures = 0;
    _resetIdleTimer(); // 启动播放时激活计时器
    if (_currentFilePath != null) {
      // 恢复暂停：重播当前文件（pause 时用 stop 释放了播放器，无法 resume）
      // 无论缓冲区是否有内容，都先把当前句插入队首，避免跳句
      _buffer.prepend(
        BufferedAudio(
          filePath: _currentFilePath!,
          lineIndex: _currentItem?.lineIndex ?? -1,
          endLineIndex: _currentItem?.endLineIndex ?? -1,
          text: _currentItem?.text ?? '',
          title: _currentItem?.title ?? '',
          session: _session,
        ),
```

```
      );
      _startPump();
      _applyState(
        TtsAudioPlaying(
          item: snapshotOf(_currentItem!),
          bufferedCount: _buffer.count,
          targetCount: _buffer.maxSize,
          playbackRate: _playbackRate,
          fallbackMessage: _fallbackMessage,
        ),
      );
    } else if (_currentItem != null) {
      // 没有文件路径但有 currentItem（可能是正在下载时暂停）
      _startPump();
    } else {
      _session++;
      _applyState(
        TtsAudioBuffering(
          bufferedCount: _buffer.count,
          targetCount: _buffer.maxSize,
          progress: _buffer.healthRatio,
          session: _session,
          playbackRate: _playbackRate,
          fallbackMessage: _fallbackMessage,
        ),
      );
      _startPump();
    }
  }
  /// 暂停播放。
  Future<void> pause() async {
    _pumpActive = false;
    _pausedGuard.mark(_currentItem);
    _isPausing = true; // 开启暂停标识，阻止 _onPlaybackComplete 推进进度
    try {
      await _engine.stopAudio();
      await _engine.pauseAudio();
    } finally {
      _isPausing = false;
    }
    _applyState(
      TtsAudioPaused(
        item: _currentItem != null ? snapshotOf(_currentItem!) : null,
        bufferedCount: _buffer.count,
        targetCount: _buffer.maxSize,
        session: _session,
        playbackRate: _playbackRate,
        fallbackMessage: _fallbackMessage,
      ),
    );
```

```
0230  }
0016  static const String _kBookshelf = 'local_bookshelf';
0017  static const String _kReadingRecords = 'reading_records';
0018  static const String _kCurrentNovelId = 'current_novel_id';
0019  static const String _kNovelIndex = 'novel_index';
0020  static const String _kSettingSound = 'setting_sound';
0021  static const String _kSettingStoryTts = 'setting_story_tts';
0022  static const String _kSettingVoice = 'setting_voice';
0023  static const String _kSettingIdleTimeout = 'setting_idle_timeout';
0024  static const String _kSettingTtsRate = 'setting_tts_rate';
0025  static const String _kSettingAmbientVol = 'setting_ambient_vol';
0026  static const String _kSettingAmbientEnabled = 'setting_ambient_enabled';
0027  static const String _kSettingAmbientStyle = 'setting_ambient_style';
0028  static const String _kHasAgreedPrivacy = 'has_agreed_privacy';
0029  static const String _kHasSelectedBook = 'user_has_selected_book';
0030  static const String _kCurrentChapterIndex = 'current_chapter_index';
0031  static const String _kSettingAnimationQuality = 'setting_animation_quality';
0033  static SharedPreferences? _prefs;
0035  static Future<void> init() async {
0036    _prefs ??= await SharedPreferences.getInstance();
0037  }
0039  /// 测试专用：清除 SharedPreferences 单例缓存，使 [init] 可重新初始化
0040  /// 生产环境禁止调用
0041  @visibleForTesting
0042  static void resetForTesting() => _prefs = null;
0044  /// SharedPreferences 单例访问入口。
0045  ///
0046  /// P0-2：release 模式下 `assert` 会被擦除，原代码紧跟的 `_prefs!` 会抛
0047  /// 不带语义的 `Null check operator used on a null value`，无法 fail-fast 定位。
0048  /// 改为显式 `StateError`，确保 release 包未初始化访问时崩溃信息清晰。
0049  static SharedPreferences get _p {
0050    final prefs = _prefs;
0051    if (prefs == null) {
0052      throw StateError(
0053        'StorageService.init() 必须在 runApp 之前调用，'
0054        '未初始化时禁止访问 SharedPreferences。',
0055      );
0056    }
0057    return prefs;
0058  }
0060  // ─── 游戏状态 (local_save_data) ──────────────────────────────────────────────────────────────
0061  /// 对应 JS saveLocalState() 的完整快照
0062  static Future<void> saveGameState({
0063    required List<List<Map<String, dynamic>?>> board,
0064    required int score,
0065    required int combo,
0066    required int bestScore,
0067    required int maxCombo,
0068    required int novelIndex,
0069    String? currentNovelId,
```

```
  }) async {
    final st = {
      'board_data': jsonEncode(board),
      'score': score,
      'combo': combo,
      'bestScore': bestScore,
      'maxCombo': maxCombo,
      'novel_index': novelIndex,
      'current_novel_id': currentNovelId,
    };
    await _p.setString(_kLocalSave, jsonEncode(st));
    await _p.setInt(_kBestScore, bestScore);
    await _p.setInt(_kMaxCombo, maxCombo);
    await _p.setInt(_kNovelIndex, novelIndex);
    // P1-7：currentNovelId 为 null 时显式删除键，避免删书后旧 id 残留 prefs。
    if (currentNovelId != null) {
      await _p.setString(_kCurrentNovelId, currentNovelId);
    } else {
      await _p.remove(_kCurrentNovelId);
    }
  }
  static Map<String, dynamic>? loadGameState() {
    final saved = _p.getString(_kLocalSave);
    if (saved == null) return null;
    try {
      return jsonDecode(saved) as Map<String, dynamic>;
    } catch (e, st) {
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'game',
        extra: {'context': 'loadGameState JSON 解析失败，本地存档已损坏'},
      );
      return null;
    }
  }
  static int loadBestScore() => _p.getInt(_kBestScore) ?? 0;
  static int loadMaxCombo() => _p.getInt(_kMaxCombo) ?? 0;
  // ─── 书架元数据 (local_bookshelf) ──────────────────────────────────────────────
  static Future<void> saveBookshelf(List<Map<String, dynamic>> shelf) async {
    await _p.setString(_kBookshelf, jsonEncode(shelf));
  }
  static List<Map<String, dynamic>> loadBookshelf() {
    final s = _p.getString(_kBookshelf);
    if (s == null) return [];
    try {
      return (jsonDecode(s) as List).cast<Map<String, dynamic>>();
    } catch (e, st) {
      CyberLogger.captureWarning(
        e,
```

```
        stack: st,
        tag: 'library',
        extra: {'context': 'loadBookshelf JSON 解析失败，书架数据已损坏'},
      );
      return [];
    }
  }
  // ─── 阅读进度 (reading_records / ProgressManager) ──────────────────────────
  static Future<void> updateReadingRecord(
    String bookId,
    int cursor,
    int total,
  ) async {
    if (total <= 0) return;
    final records = _loadReadingRecords();
    final percent = (cursor / total * 100).clamp(0.0, 100.0);
    records[bookId] = {
      'cursor': cursor,
      'total': total,
      'percent': double.parse(percent.toStringAsFixed(2)),
    };
    await _p.setString(_kReadingRecords, jsonEncode(records));
  }
  static Map<String, dynamic> getReadingRecord(String bookId) {
    final records = _loadReadingRecords();
    return records[bookId] as Map<String, dynamic>? ??
        {'cursor': 0, 'total': 1, 'percent': 0.0};
  }
  static Future<void> deleteReadingRecord(String bookId) async {
    final records = _loadReadingRecords();
    records.remove(bookId);
    await _p.setString(_kReadingRecords, jsonEncode(records));
  }
  static Map<String, dynamic> _loadReadingRecords() {
    final s = _p.getString(_kReadingRecords);
    if (s == null) return {};
    try {
      return (jsonDecode(s) as Map).cast<String, dynamic>();
    } catch (e, st) {
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'reader',
        extra: {'context': '_loadReadingRecords JSON 解析失败，进度记录已损坏'},
      );
      return {};
    }
  }
  static Future<void> setCurrentNovelId(String? id) async {
    if (id == null) {
```

```
0179      await _p.remove(_kCurrentNovelId);
0180    } else {
0181      await _p.setString(_kCurrentNovelId, id);
0182    }
0183  }
0185  static String? getCurrentNovelId() => _p.getString(_kCurrentNovelId);
0187  static int getCurrentNovelIndex() => _p.getInt(_kNovelIndex) ?? 0;
0189  static int getCurrentChapterIndex() => _p.getInt(_kCurrentChapterIndex) ?? 0;
0191  static Future<void> setCurrentChapterIndex(int index) async {
0192    await _p.setInt(_kCurrentChapterIndex, index);
0193  }
0195  static Future<void> setCurrentNovelIndex(int index) async {
0196    await _p.setInt(_kNovelIndex, index);
0197  }
0199  // ─── 书籍正文内容（替代 IndexedDB LocalDB.js）──────────────────────────────
0200  static Future<File> _bookFile(String bookId) async {
0201    final dir = await getApplicationDocumentsDirectory();
0202    return File('${dir.path}/books/$bookId.json');
0203  }
0205  static Future<void> saveBookContent(
0206    String bookId, {
0207    required List<String> lines,
0208    required List<Map<String, dynamic>> chapters,
0209  }) async {
0210    try {
0211      final file = await _bookFile(bookId);
0212      await file.parent.create(recursive: true);
0213      await file
0214          .writeAsString(jsonEncode({'lines': lines, 'chapters': chapters}));
0215    } catch (e, st) {
0216      CyberLogger.captureWarning(
0217        e,
0218        stack: st,
0219        tag: 'library',
0220        extra: {'context': 'StorageService.saveBookContent 失败'},
0221      );
0222    }
0223  }
0225  static Future<Map<String, dynamic>?> loadBookContent(String bookId) async {
0226    try {
0227      final file = await _bookFile(bookId);
0228      if (!await file.exists()) return null;
0229      final content = await file.readAsString();
0230      if (content.trim().isEmpty) return null;
0231      return jsonDecode(content) as Map<String, dynamic>;
0232    } catch (e, st) {
0233      CyberLogger.captureWarning(
0234        e,
0235        stack: st,
0236        tag: 'library',
```

```
        extra: {'context': 'StorageService.loadBookContent 失败'},
      );
      return null;
    }
  }
  static Future<void> deleteBookContent(String bookId) async {
    try {
      final file = await _bookFile(bookId);
      if (await file.exists()) await file.delete();
    } catch (e, st) {
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'library',
        extra: {'context': 'deleteBookContent 文件删除失败', 'bookId': bookId},
      );
    }
  }
  // ── 全局设置 ──────────────────────────────────────────────────────────────────────────────────────────────────────
  static bool getSettingSound() => _p.getBool(_kSettingSound) ?? true;
  static Future<void> setSettingSound(bool v) => _p.setBool(_kSettingSound, v);
  static bool getSettingStoryTts() => _p.getBool(_kSettingStoryTts) ?? true;
  static Future<void> setSettingStoryTts(bool v) =>
      _p.setBool(_kSettingStoryTts, v);
  static String getSettingVoice() =>
      _p.getString(_kSettingVoice) ?? 'zh-CN-XiaoxiaoNeural';
  static Future<void> setSettingVoice(String v) =>
      _p.setString(_kSettingVoice, v);
  static int getSettingIdleTimeout() => _p.getInt(_kSettingIdleTimeout) ?? 1;
  static Future<void> setSettingIdleTimeout(int v) =>
      _p.setInt(_kSettingIdleTimeout, v);
  static double getSettingTtsRate() => _p.getDouble(_kSettingTtsRate) ?? 1.0;
  static Future<void> setSettingTtsRate(double v) =>
      _p.setDouble(_kSettingTtsRate, v);
  static double getSettingAmbientVol() =>
      _p.getDouble(_kSettingAmbientVol) ?? 0.5;
  static Future<void> setSettingAmbientVol(double v) =>
      _p.setDouble(_kSettingAmbientVol, v);
  static bool getSettingAmbientEnabled() =>
      _p.getBool(_kSettingAmbientEnabled) ?? true;
  static Future<void> setSettingAmbientEnabled(bool v) =>
      _p.setBool(_kSettingAmbientEnabled, v);
  static String getSettingAmbientStyle() =>
      _p.getString(_kSettingAmbientStyle) ?? 'wuxia';
  static Future<void> setSettingAmbientStyle(String v) =>
      _p.setString(_kSettingAmbientStyle, v);
  static String getSettingAnimationQuality() =>
      _p.getString(_kSettingAnimationQuality) ?? 'auto';
  static Future<void> setSettingAnimationQuality(String v) =>
      _p.setString(_kSettingAnimationQuality, v);
```

```
  // ─── 隐私协议同意状态 ──────────────────────────────────────────────────────────────────────────────────────────────
  static bool hasAgreedPrivacy() => _p.getBool(_kHasAgreedPrivacy) ?? false;
  static Future<void> setHasAgreedPrivacy(bool v) =>
      _p.setBool(_kHasAgreedPrivacy, v);
  // ─── 粘性位：用户是否曾主动选择过书籍 ──────────────────────────────────────────────────────────────────────────────
  static bool hasSelectedBook() => _p.getBool(_kHasSelectedBook) ?? false;
  static Future<void> setHasSelectedBook(bool v) =>
      _p.setBool(_kHasSelectedBook, v);
  // ─── 分章文本缓存（文件路径，非 SharedPreferences）──────────────────────────────────────────────────────────────
  static Future<File> _chapterCacheFile(String bookId, int chapterIndex) async {
    final dir = await getApplicationDocumentsDirectory();
    final idx = chapterIndex.toString().padLeft(3, '0');
    return File('${dir.path}/books/chapters/$bookId/$idx.txt');
  }
  static Future<Directory> _chapterCacheDir(String bookId) async {
    final dir = await getApplicationDocumentsDirectory();
    return Directory('${dir.path}/books/chapters/$bookId');
  }
  static Future<void> saveChapterCache(
    String bookId,
    int chapterIndex,
    String text,
  ) async {
    try {
      final file = await _chapterCacheFile(bookId, chapterIndex);
      await file.parent.create(recursive: true);
      await file.writeAsString(text);
    } catch (e, st) {
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'library',
        extra: {'context': 'StorageService.saveChapterCache 失败'},
      );
    }
  }
  static Future<String?> loadChapterCache(
    String bookId,
    int chapterIndex,
  ) async {
    try {
      final file = await _chapterCacheFile(bookId, chapterIndex);
      if (!await file.exists()) return null;
      final content = await file.readAsString();
      return content.trim().isEmpty ? null : content;
    } catch (e, st) {
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'library',
```

```
        extra: {'context': 'StorageService.loadChapterCache 失败'},
      );
      return null;
    }
  }
  static Future<void> clearChapterCache(String bookId) async {
    try {
      final dir = await _chapterCacheDir(bookId);
      if (await dir.exists()) await dir.delete(recursive: true);
    } catch (e, st) {
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'library',
        extra: {'context': 'StorageService.clearChapterCache 失败'},
      );
    }
  }
  /// LRU 清理：保留 [currentChapterIndex ± keepAround] 范围内文件，其余删除。
  static Future<void> pruneChapterCache(
    String bookId,
    int currentChapterIndex, {
    int keepAround = 3,
  }) async {
    try {
      final dir = await _chapterCacheDir(bookId);
      if (!await dir.exists()) return;
      final keepMin = currentChapterIndex - keepAround;
      final keepMax = currentChapterIndex + keepAround;
      await for (final entity in dir.list()) {
        if (entity is! File) continue;
        final name = entity.uri.pathSegments.last;
        if (!name.endsWith('.txt')) continue;
        final idx = int.tryParse(name.replaceFirst('.txt', ''));
        if (idx == null) continue;
        if (idx < keepMin || idx > keepMax) await entity.delete();
      }
    } catch (e, st) {
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'library',
        extra: {'context': 'StorageService.pruneChapterCache 失败'},
      );
    }
  }
  // ─── 书目录缓存（文件路径）──────────────────────────────────────────────────
  static Future<File> _catalogFile(String bookId) async {
    final dir = await getApplicationDocumentsDirectory();
    return File('${dir.path}/books/catalogs/${bookId}_catalog.json');
```

```
0406    }
0408    static Future<void> saveBookCatalog(
0409      String bookId,
0410      List<Map<String, dynamic>> chapters,
0411    ) async {
0412      try {
0413        final file = await _catalogFile(bookId);
0414        await file.parent.create(recursive: true);
0415        await file.writeAsString(jsonEncode(chapters));
0416      } catch (e, st) {
0417        CyberLogger.captureWarning(
0418          e,
0419          stack: st,
0420          tag: 'library',
0421          extra: {'context': 'StorageService.saveBookCatalog 失败'},
0422        );
0423      }
0424    }
0426    static Future<List<Map<String, dynamic>>?> loadBookCatalog(
0427      String bookId,
0428    ) async {
0429      try {
0430        final file = await _catalogFile(bookId);
0431        if (!await file.exists()) return null;
0432        final content = await file.readAsString();
0433        if (content.trim().isEmpty) return null;
0434        return (jsonDecode(content) as List).cast<Map<String, dynamic>>();
0435      } catch (e, st) {
0436        CyberLogger.captureWarning(
0437          e,
0438          stack: st,
0439          tag: 'library',
0440          extra: {'context': 'StorageService.loadBookCatalog 失败'},
0441        );
0442        return null;
0443      }
0444    }
0445  }
// ===== 文件结束：lib/core/database/storage_service.dart =====

// ===== 文件：lib/main.dart =====
0001  import 'dart:ui';
0002  import 'package:flutter/material.dart';
0003  import 'package:flutter/services.dart';
0004  import 'core/constants/book_constants.dart';
0005  import 'core/database/storage_service.dart';
0006  import 'core/theme/cyber_colors.dart';
0007  import 'core/utils/cyber_logger.dart';
0008  import 'features/settings/presentation/widgets/privacy_agreement_modal.dart';
0009  import 'features/audio/services/ambient_service.dart';
```

```
import 'features/audio/services/sfx_service.dart';
import 'features/dashboard/presentation/dashboard_screen.dart';
import 'features/library/domain/book_model.dart';
import 'features/reader/providers/reader_provider.dart';
import 'features/settings/providers/settings_provider.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart' as riverpod;
import 'shared/widgets/tts_error_listener.dart';
import 'features/audio/services/tts_engine_service.dart';
import 'features/audio/providers/tts_audio_notifier.dart';
final GlobalKey<NavigatorState> globalNavigatorKey =
    GlobalKey<NavigatorState>();
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  // 全局错误捕获锚点（先注册，确保 Sentry 初始化前的错误也能记录）
  FlutterError.onError = CyberLogger.recordFlutterError;
  PlatformDispatcher.instance.onError = CyberLogger.recordPlatformError;
  // 启动前初始化持久化层与音效引擎
  await StorageService.init();
  try {
    await SfxService.init();
  } catch (e, st) {
    CyberLogger.recordFlutterError(
      FlutterErrorDetails(exception: e, stack: st, library: 'SfxService'),
    );
  }
  // 初始化环境背景音服务
  try {
    await AmbientService.init();
  } catch (e, st) {
    CyberLogger.recordFlutterError(
      FlutterErrorDetails(exception: e, stack: st, library: 'AmbientService'),
    );
  }
  // 通过 CyberLogger 初始化 Sentry 并启动 App
  // DSN 通过 --dart-define=SENTRY_DSN=https://... 注入，空时静默跳过
  await CyberLogger.initSentry(
    () async => runApp(
      const riverpod.ProviderScope(
        child: YueYouApp(),
      ),
    ),
  );
}
class YueYouApp extends riverpod.ConsumerWidget {
  const YueYouApp({super.key});
  @override
  Widget build(BuildContext context, riverpod.WidgetRef ref) {
    return const _Bootstrapper();
  }
}
```

```
0069 class _Bootstrapper extends riverpod.ConsumerStatefulWidget {
0070   const _Bootstrapper();
0072   @override
0073   riverpod.ConsumerState<_Bootstrapper> createState() => _BootstrapperState();
0074 }
0076 class _BootstrapperState extends riverpod.ConsumerState<_Bootstrapper>
0077     with WidgetsBindingObserver {
0078   bool _booted = false;
0080   /// 记录上一次 SettingsProvider 的 ambient 状态，用于增量对比
0081   bool? _lastAmbientEnabled;
0082   double? _lastAmbientVol;
0083   String? _lastAmbientStyle;
0085   @override
0086   void initState() {
0087     super.initState();
0088     WidgetsBinding.instance
0089       ..addObserver(this)
0090       ..addPostFrameCallback((_) => _checkPrivacyAndBootstrap());
0091   }
0093   @override
0094   void didChangeDependencies() {
0095     super.didChangeDependencies();
0096     // 每当 SettingsProvider 变化时同步 AmbientService
0097     final settings = ref.watch(settingsProvider);
0098     _syncAmbient(settings);
0099   }
0101   /// 同步 SettingsProvider 的 ambient 设置到 AmbientService
0102   void _syncAmbient(SettingsProvider settings) {
0103     if (settings.ambientEnabled != _lastAmbientEnabled) {
0104       _lastAmbientEnabled = settings.ambientEnabled;
0105       AmbientService.setEnabled(settings.ambientEnabled);
0106     }
0107     if (settings.ambientVol != _lastAmbientVol) {
0108       _lastAmbientVol = settings.ambientVol;
0109       AmbientService.setVolume(settings.ambientVol);
0110     }
0111     if (settings.ambientStyle != _lastAmbientStyle) {
0112       _lastAmbientStyle = settings.ambientStyle;
0113       AmbientService.setStyle(settings.ambientStyle);
0114     }
0115   }
0117   /// App 生命周期：切后台暂停，切前台恢复。
0118   ///
0119   /// P0-3：原实现把 `hidden` 与 `detached` 一并 `dispose()`，但 Android `hidden`
0120   /// 是瞬态状态（电源键熄屏、最近任务、画中画切换），dispose 后 `_initialized=false`
0121   /// 会让本次进程内 AmbientService 永久哑火。改为：
0122   /// - paused / inactive / hidden ：均仅 `pause()`，资源保留以便快速恢复；
0123   /// - detached ：仅在进程将被销毁时才 `dispose()`。
0124   @override
0125   void didChangeAppLifecycleState(AppLifecycleState state) {
```

```
    switch (state) {
      case AppLifecycleState.paused:
      case AppLifecycleState.inactive:
      case AppLifecycleState.hidden:
        AmbientService.pause();
        ref.read(ttsAudioProvider.notifier).setBackgroundTolerant(true);
      case AppLifecycleState.resumed:
        AmbientService.resume();
        ref.read(ttsAudioProvider.notifier).setBackgroundTolerant(false);
      case AppLifecycleState.detached:
        AmbientService.dispose();
    }
  }
  Future<void> _checkPrivacyAndBootstrap() async {
    if (!mounted) return;
    if (!StorageService.hasAgreedPrivacy()) {
      // 等待 MaterialApp 内部 Navigator 挂载完成（最多 5 次，间隔 50ms）
      // 避免首帧时 globalNavigatorKey 尚未绑定导致弹窗被静默跳过
      BuildContext? navContext;
      for (int i = 0; i < 5; i++) {
        navContext = globalNavigatorKey.currentContext;
        if (navContext != null) break;
        await Future<void>.delayed(const Duration(milliseconds: 50));
        if (!mounted) return;
      }
      if (navContext == null) {
        // 极端情况：Navigator 始终未挂载，记录日志后退出，绝不进入未授权状态
        CyberLogger.captureWarning(
          Exception('隐私弹窗 navContext 重试 5 次仍为 null'),
          tag: 'privacy',
          extra: {'context': '_checkPrivacyAndBootstrap'},
        );
        SystemNavigator.pop();
        return;
      }
      // navContext 来自 globalNavigatorKey，由 Navigator 自身管理生命周期，
      // 不依赖 _BootstrapperState.mounted；上方循环已做 mounted 守卫。
      // ignore: use_build_context_synchronously
      final agreed = await showPrivacyAgreementModal(navContext);
      if (!mounted) return;
      if (agreed) {
        await StorageService.setHasAgreedPrivacy(true);
      } else {
        return;
      }
    }
    await _bootstrap();
  }
  Future<void> _bootstrap() async {
    if (_booted) return;
```

```
    _booted = true;
    final reader = ref.read(readerProvider);
    final currentNovelId = StorageService.getCurrentNovelId();
    if (currentNovelId == null) return;
    // P1-8：默认书《西游记》走分章懒加载路径，由 readerProvider 创建时
    // 触发的 restoreDefaultBook() 独占恢复，_bootstrap 不再读其全书内容，
    // 避免与 microtask 形成竞态把空数据写入 _sentences。
    if (currentNovelId == BookConstants.defaultBookKey) return;
    final content = await StorageService.loadBookContent(currentNovelId);
    if (!mounted || content == null) return;
    final List<dynamic> rawLines = content['lines'] as List<dynamic>? ?? [];
    final List<dynamic> rawChapters =
        content['chapters'] as List<dynamic>? ?? [];
    final chapters = rawChapters
        .map((e) => ChapterModel.fromJson(e as Map<String, dynamic>))
        .toList();
    final lines = rawLines.map((e) => e.toString()).toList();
    final int initialIndex = StorageService.getCurrentNovelIndex();
    await reader.loadPreparedBook(
      lines,
      bookId: currentNovelId,
      chapters: chapters,
      initialIndex: initialIndex,
    );
  }
  @override
  void dispose() {
    WidgetsBinding.instance.removeObserver(this);
    super.dispose();
  }
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      navigatorKey: globalNavigatorKey,
      title: '阅游 YueYou',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        scaffoldBackgroundColor: CyberColors.background,
        colorScheme: const ColorScheme.dark(
          primary: CyberColors.neonGreen,
          secondary: CyberColors.neonPink,
        ),
        useMaterial3: true,
      ),
      builder: (context, child) => Listener(
        onPointerDown: (_) => ref.read(ttsEngineProvider).notifyUserActivity(),
        behavior: HitTestBehavior.translucent,
        child: TtsErrorListener(child: child!),
      ),
      home: const DashboardScreen(),
```

```
0234      );
0235    }
0236  }
// ===== 文件结束：lib/main.dart =====


// ===== 文件：lib/features/reader/providers/reader_provider.dart =====
0001  import 'dart:async';
0003  import 'package:flutter/foundation.dart';
0004  import 'package:yueyou/core/utils/cyber_logger.dart';
0005  import 'package:yueyou/core/constants/book_constants.dart';
0006  import 'package:yueyou/core/utils/text_processing.dart';
0007  import 'package:yueyou/core/database/storage_service.dart';
0008  import 'package:yueyou/features/audio/providers/tts_audio_notifier.dart';
0009  import 'package:yueyou/features/library/services/default_book_service.dart';
0010  import 'package:yueyou/features/reader/domain/chapter_load_state.dart';
0011  import 'package:yueyou/features/reader/domain/reader_sentence_cursor.dart';
0012  import 'package:yueyou/features/reader/domain/text_parser.dart';
0013  import 'package:yueyou/features/audio/services/tts_engine_service.dart';
0014  import 'package:yueyou/features/library/domain/book_model.dart';
0015  import 'package:flutter_riverpod/flutter_riverpod.dart';
0017  /// 阻读器全局 Provider（Riverpod 生命周期托管版）
0018  ///
0019  /// - 通过 [ref.read] 获取 [ttsEngineProvider] 用于影子状态监听
0020  /// - 通过 [ref.read] 获取 [ttsAudioProvider.notifier] 注册句子源
0021  /// - 通过 [ref.onDispose] 自动回收阻读器资源
0022  final readerProvider = ChangeNotifierProvider<ReaderProvider>((ref) {
0023    final engine = ref.read(ttsEngineProvider);
0024    final notifier = ref.read(ttsAudioProvider.notifier);
0025    final rp = ReaderProvider(engine, notifier: notifier);
0026    notifier.registerSentenceSource(rp);
0027    ref.onDispose(rp.dispose);
0028    // 热重启恢复：若上次阅读的是默认书籍，自动重载当前章节
0029    final lastNovelId = StorageService.getCurrentNovelId();
0030    if (lastNovelId == BookConstants.defaultBookKey) {
0031      Future.microtask(() => rp.restoreDefaultBook());
0032    }
0033    return rp;
0034  });
0036  /// TTS 切换操作的领域结果
0037  enum TtsToggleResult {
0038    /// 成功切换到播放
0039    playing,
0041    /// 成功切换到暂停
0042    paused,
0044    /// 无有效书籍数据，无法开启 TTS
0045    noContent,
0046  }
0048  /// 阅游提词器 Provider
0049  /// 职责：管理 sentences 和 currentIndex，并联动 TtsEngineService 进行语音播报
0050  class ReaderProvider with ChangeNotifier implements TtsSentenceSource {
```

```
0051    final TtsEngineService _ttsEngine;
0052    final TtsAudioNotifier? _ttsNotifier;
0053    final Future<ParseResult> Function(String rawText) _parseBook;
0054    List<String> _sentences = [];
0055    int _currentIndex = 0;
0056    int _fetchIndex = 0;
0057    bool _isParsing = false;
0058    String? _currentBookId;
0059    List<ChapterModel> _chapters = [];
0060    String? _lastTtsError;
0061    TtsPlaybackState _lastTtsState = TtsPlaybackState.disabled;
0063    // ─── 分章懒加载（默认书籍模式）──────────────────────────────────────────────────────────────
0064    int? _currentChapterIndex;
0065    ChapterLoadState _chapterLoadState = ChapterLoadState.idle;
0066    bool _isDefaultBookMode = false;
0067    DefaultBookService? _defaultBookService;
0069    /// 噪音/空行判定（TTS 和游标均应跳过，绝不朗读）。
0070    /// 句子合并算法已抽到 `reader_sentence_cursor.dart` 直接使用
0071    /// [TextProcessing.isNoiseLine]；本 wrapper 仅服务于 `jumpTo` 内部跳过
0072    /// 噪音行的细节流程。
0073    bool _isNoise(String text) => TextProcessing.isNoiseLine(text);
0075    ReaderProvider(
0076      this._ttsEngine, {
0077      TtsAudioNotifier? notifier,
0078      Future<ParseResult> Function(String rawText)? parseBook,
0079      DefaultBookService? defaultBookService,
0080    })  : _ttsNotifier = notifier,
0081          _parseBook = parseBook ?? TextParser.parse,
0082          _defaultBookService = defaultBookService {
0083      _lastTtsError = _ttsEngine.lastError;
0084      _lastTtsState = _ttsEngine.state;
0085      _ttsEngine.addListener(_onTtsEngineChanged);
0086      // 句子源回调由 TtsAudioNotifier 通过 registerSentenceSource 注册
0087    }
0089    @override
0090    Future<TtsAudioRequest?> nextTtsSentence(int session) async {
0091      final result = fetchNextSentenceRequest(
0092        sentences: _sentences,
0093        fetchIndex: _fetchIndex,
0094        chapterTitle: currentChapterTitle,
0095      );
0096      _fetchIndex = result.nextFetchIndex;
0097      return result.request;
0098    }
0100    /// 播放开始时，用真实 lineIndex 同步 UI。
0101    @override
0102    FutureOr<void> onTtsItemStarted(TtsAudioItem item) {
0103      final session = _ttsNotifier?.currentSession;
0104      // 无编排层时跳过 session 校验（向下兼容）
0105      if (session != null && item.session != session) return null;
```

```
    if (_currentIndex != item.lineIndex) {
      _currentIndex = item.lineIndex;
      notifyListeners();
    }
  }
  /// 播放完成时只更新 UI 游标，不修改预加载游标。
  @override
  Future<void> onTtsItemFinished(TtsAudioItem item) async {
    final session = _ttsNotifier?.currentSession;
    // 无编排层时跳过 session 校验（向下兼容）
    if (session != null && item.session != session) return;
    if (_sentences.isEmpty) return;
    //  自动步进：从「合并段最后一行」之后开始扫描，跳过噪音行
    // 关键：item.endLineIndex 已包含本句合并消耗的所有行，
    // 使用它推进可保证提词器不跳行、与 TTS 进度严格对齐。
    int nextIdx = item.endLineIndex + 1;
    if (nextIdx <= _currentIndex) nextIdx = _currentIndex + 1;
    while (nextIdx < _sentences.length && _isNoise(_sentences[nextIdx])) {
      nextIdx++;
    }
    if (nextIdx < _sentences.length) {
      _currentIndex = nextIdx;
      //  重要修复：此处绝不可重置 _fetchIndex！
      // 预取指针应当保持领先，重置它会导致预取循环重新抓取已在队列中的任务。
      notifyListeners();
    } else {
      // 章节末尾：先钉住 UI 游标到当前句位置
      if (item.lineIndex >= 0 && item.lineIndex < _sentences.length) {
        _currentIndex = item.lineIndex;
        notifyListeners();
      }
      //  章节末尾 + 默认书籍模式：自动推进到下一章
      // 注意：必须与上方 UI 更新分离（不可用 else if），二者应同时执行
      if (_isDefaultBookMode) {
        _autoAdvanceChapter();
      }
    }
    _saveProgress().catchError((e) {
      CyberLogger.captureWarning(
        e,
        tag: 'reader',
        extra: {'context': 'onTtsItemFinished 进度保存失败'},
      );
    });
  }
  /// 重置预取游标到当前阅读位置。
  @override
  void resetFetchIndex() {
    _fetchIndex = _currentIndex;
  }
```

```
0162    List<String> get sentences => _sentences;
0163    int get currentIndex => _currentIndex;
0164    int get fetchIndex => _fetchIndex;
0165    bool get isParsing => _isParsing;
0166    TtsEngineService get ttsEngine => _ttsEngine;
0167    List<ChapterModel> get chapters => _chapters;
0168    String? get currentBookId => _currentBookId;
0169    String? get ttsErrorMessage => _lastTtsError;
0171    // ─── 分章懒加载 getter ──────────────────────────────────────────────────────────────────────────────────────────────────────
0172    ChapterLoadState get chapterLoadState => _chapterLoadState;
0173    bool get isDefaultBookMode => _isDefaultBookMode;
0174    int? get currentChapterIndex => _currentChapterIndex;
0176    /// 清理当前 TTS 错误提示（例如 UI 已展示并确认后）。
0177    void clearTtsError() {
0178      _ttsEngine.clearLastError();
0179    }
0181    void _onTtsEngineChanged() {
0182      bool changed = false;
0183      final nextError = _ttsEngine.lastError;
0184      if (nextError != _lastTtsError) {
0185        _lastTtsError = nextError;
0186        changed = true;
0187      }
0189      final nextState = _ttsEngine.state;
0190      if (nextState != _lastTtsState) {
0191        _lastTtsState = nextState;
0192        changed = true;
0193      }
0195      if (changed) {
0196        notifyListeners();
0197      }
0198    }
0200    /// 计算当前章节标题（遍历 chapters，找到 lineIndex <= currentIndex 的最新章节）
0201    String get currentChapterTitle {
0202      if (_chapters.isEmpty) return '未知章节';
0204      ChapterModel? currentChapter;
0205      for (final chapter in _chapters) {
0206        if (chapter.lineIndex <= _currentIndex) {
0207          currentChapter = chapter;
0208        } else {
0209          break;
0210        }
0211      }
0213      final raw = currentChapter?.title ?? _chapters.first.title;
0214      return TextProcessing.cleanChapterTitle(raw);
0215    }
0217    /// 进度引擎：当前进度百分比 (0.01 - 1.0)
0218    double get progress {
0219      if (_sentences.isEmpty) return 0.0;
0220      return (_currentIndex + 1) / _sentences.length;
```

```
0221  }
0223  /// 获取当前显示的句子内容
0224  String? get currentSentence {
0225    final currentItem = _ttsEngine.currentItem;
0226    if (currentItem != null &&
0227        currentItem.session == _ttsNotifier?.currentSession &&
0228        currentItem.text.trim().isNotEmpty) {
0229      return currentItem.text;
0230    }
0231    if (_sentences.isEmpty || _currentIndex >= _sentences.length) return null;
0232    return _sentences[_currentIndex];
0233  }
0235  Future<void> _applyLoadedBook({
0236    required List<String> sentences,
0237    required List<int> rawLineOrigins,
0238    String? bookId,
0239    List<ChapterModel>? chapters,
0240    int? initialIndex,
0241  }) async {
0242    _sentences = sentences;
0243    _currentBookId = bookId;
0245    // 加载非默认书时，强制清除默认书模式残留，避免章末误触 _autoAdvanceChapter
0246    if (bookId != BookConstants.defaultBookKey) {
0247      _isDefaultBookMode = false;
0248      _currentChapterIndex = null;
0249      _chapterLoadState = ChapterLoadState.idle;
0250    }
0252    final Map<int, int> rawLineToSentIdx = {};
0253    for (int i = 0; i < rawLineOrigins.length; i++) {
0254      rawLineToSentIdx.putIfAbsent(rawLineOrigins[i], () => i);
0255    }
0257    final rawChapters = chapters ?? <ChapterModel>[];
0258    _chapters = rawChapters.map((c) {
0259      final cleaned = TextProcessing.cleanChapterTitle(c.title);
0260      final mappedIdx = rawLineToSentIdx[c.lineIndex] ?? c.lineIndex;
0261      return ChapterModel(title: cleaned, lineIndex: mappedIdx);
0262    }).toList();
0264    int targetIndex = 0;
0265    if (initialIndex != null) {
0266      targetIndex = initialIndex;
0267    } else if (bookId != null) {
0268      final record = StorageService.getReadingRecord(bookId);
0269      final total = (record['total'] as num?)?.toInt() ?? 0;
0270      if (total > 1) {
0271        targetIndex = (record['cursor'] as num?)?.toInt() ?? 0;
0272      }
0273    }
0275    _currentIndex =
0276        targetIndex.clamp(0, _sentences.isEmpty ? 0 : _sentences.length - 1);
0277    _fetchIndex = _currentIndex;
```

```
    await StorageService.setCurrentNovelId(bookId);
    // 只有 TTS 正在播放/缓冲时才刷新会话（新章继续播）。
    if (_ttsNotifier?.isActivelyPlaying == true) {
      _ttsNotifier?.refreshSession();
    }
  }
  Future<void> loadPreparedBook(
    List<String> lines, {
    String? bookId,
    List<ChapterModel>? chapters,
    int? initialIndex,
  }) async {
    if (_isParsing) return;
    _isParsing = true;
    notifyListeners();
    try {
      final List<String> sentences = lines
          .map((line) => line.trim())
          .where((line) => line.isNotEmpty)
          .toList(growable: false);
      final List<int> rawLineOrigins = List<int>.generate(
        sentences.length,
        (index) => index,
        growable: false,
      );
      await _applyLoadedBook(
        sentences: sentences,
        rawLineOrigins: rawLineOrigins,
        bookId: bookId,
        chapters: chapters,
        initialIndex: initialIndex,
      );
    } catch (e, st) {
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'reader',
        extra: {'context': 'ReaderProvider.loadPreparedBook 异常'},
      );
    } finally {
      _isParsing = false;
      notifyListeners();
    }
  }
  /// 核心加载方法：加载书籍并解析
  Future<void> loadBook(
    String rawText, {
    String? bookId,
    List<ChapterModel>? chapters,
    int? initialIndex,
```

```
    bool forceIndex = false,
  }) async {
    if (_isParsing) return;
    _isParsing = true;
    notifyListeners();
    try {
      final parseResult = await _parseBook(rawText);
      await _applyLoadedBook(
        sentences: parseResult.sentences,
        rawLineOrigins: parseResult.rawLineOrigins,
        bookId: bookId,
        chapters: chapters,
        initialIndex: initialIndex,
      );
    } catch (e, st) {
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'reader',
        extra: {
          'context': 'ReaderProvider.loadBook 异常',
          'bookId': bookId ?? '',
        },
      );
    } finally {
      _isParsing = false;
      notifyListeners();
    }
  }
  /// 切换播放/暂停状态
  /// 返回领域结果，由 UI 层决定如何展示提示
  ///
  /// 使用 Dart 3 穷尽 switch 表达式，确保 [TtsPlaybackState] 所有
  /// 状态分支在编译期得到穷尽性检查，消除静默失败风险。
  TtsToggleResult toggleTTS() {
    // 前置拦截——未导入书籍或数据为空时，禁止触发 TTS
    if (_currentBookId == null || _sentences.isEmpty) {
      return TtsToggleResult.noContent;
    }
    // 穷尽 switch：覆盖 TtsPlaybackState 全部 5 个分支
    final notifier = _ttsNotifier;
    if (notifier == null) return TtsToggleResult.noContent;
    return switch (_ttsEngine.state) {
      TtsPlaybackState.playing || TtsPlaybackState.buffering => () {
          notifier.pause();
          return TtsToggleResult.paused;
        }(),
      TtsPlaybackState.paused || TtsPlaybackState.disabled => () {
          notifier.play();
          return TtsToggleResult.playing;
```

```
        }(),
      TtsPlaybackState.error => () {
          _ttsEngine.clearLastError();
          notifier.play();
          return TtsToggleResult.playing;
        }(),
    };
  }
  /// 倍速切换桥接
  void cycleSpeed() {
    _ttsEngine.cycleSpeed();
  }
  /// 步进逻辑 - 进入下一句（极限性能优化版）
  Future<void> nextSentence() async {
    if (_currentIndex < _sentences.length - 1) {
      _currentIndex++;
      _fetchIndex = _currentIndex;
      notifyListeners();
      // Fire-and-forget：进度存档不阻塞主线程
      _saveProgress().catchError((e) {
        CyberLogger.captureWarning(
          e,
          tag: 'reader',
          extra: {'context': 'nextSentence 进度保存失败'},
        );
      });
      // 仅在 TTS 播放/缓冲时刷新，空闲不启泵
      if (_ttsNotifier?.isActivelyPlaying == true) {
        Future.microtask(() => _ttsNotifier?.refreshSession());
      }
    }
  }
  /// 步进逻辑 - 回退上一句（极限性能优化版）
  Future<void> previousSentence() async {
    if (_currentIndex > 0) {
      _currentIndex--;
      _fetchIndex = _currentIndex;
      notifyListeners();
      // Fire-and-forget：进度存档不阻塞主线程
      _saveProgress().catchError((e) {
        CyberLogger.captureWarning(
          e,
          tag: 'reader',
          extra: {'context': 'previousSentence 进度保存失败'},
        );
      });
      // 仅在 TTS 播放/缓冲时刷新，空闲不启泵
      if (_ttsNotifier?.isActivelyPlaying == true) {
        Future.microtask(() => _ttsNotifier?.refreshSession());
      }
```

```
    }
  }
  /// 按行号跳转（对应 JS jumpTo(lineIndex)）
  Future<void> jumpToLine(int index) => jumpTo(index);
  ///  切章核心：严格边界检查 + 智能跳过标题 + 同步UI + 安全重启TTS
  Future<void> jumpTo(int index) async {
    // 严格边界检查
    if (_sentences.isEmpty) return;
    if (index < 0 || index >= _sentences.length) {
      CyberLogger.captureWarning(
        StateError('jumpTo 越界'),
        tag: 'reader',
        extra: {'index': '$index', 'max': '${_sentences.length - 1}'},
      );
      return;
    }
    // 智能跳过噪音和空行，保留章节标题（让 TTS 从标题开始朗读）
    int targetIndex = index;
    int attempts = 0;
    while (attempts < 20 && targetIndex < _sentences.length) {
      if (!_isNoise(_sentences[targetIndex].trim())) break;
      targetIndex++;
      attempts++;
    }
    if (targetIndex >= _sentences.length) {
      targetIndex = index; // fallback
    }
    //  第一时间同步更新，让所有 UI 瞬间响应
    _currentIndex = targetIndex;
    _fetchIndex = targetIndex;
    notifyListeners();
    // Fire-and-forget 存档
    _saveProgress().catchError((e) {
      CyberLogger.captureWarning(
        e,
        tag: 'reader',
        extra: {'context': 'jumpTo 进度保存失败'},
      );
    });
    // 仅在 TTS 播放/缓冲时刷新，空闲不启泵
    if (_ttsNotifier?.isActivelyPlaying == true) {
      Future.microtask(() => _ttsNotifier?.refreshSession());
    }
  }
  /// 持久化存档逻辑（对应 JS ProgressManager.updateRecord）
  Future<void> _saveProgress() async {
    if (_currentBookId == null || _sentences.isEmpty) return;
    await StorageService.updateReadingRecord(
      _currentBookId!,
      _currentIndex,
```

```
      _sentences.length,
    );
    await StorageService.setCurrentNovelIndex(_currentIndex);
  }
  void switchChapter(int chapterIndex) {
    if (_chapters.isEmpty) return;
    if (chapterIndex < 0 || chapterIndex >= _chapters.length) return;
    _currentIndex = _chapters[chapterIndex].lineIndex;
    _fetchIndex = _chapters[chapterIndex].lineIndex;
    // 仅在 TTS 播放/缓冲时刷新，空闲不启泵
    if (_ttsNotifier?.isActivelyPlaying == true) {
      _ttsNotifier?.refreshSession();
    }
    notifyListeners();
    _saveProgress().catchError((e) {
      CyberLogger.captureWarning(
        e,
        tag: 'reader',
        extra: {'context': 'switchChapter 进度保存失败'},
      );
    });
  }
  ///  任务 1.3：级联重置——当当前正在阅读的书籍被删除时，完全重置阅读器状态
  void resetForDeletedBook(String bookId) {
    if (_currentBookId != bookId) return;
    // 停止 TTS 播放（无论是否启用，强制清空缓冲区和泵）
    _ttsNotifier?.stopAll();
    // 置空所有数据
    _currentBookId = null;
    _sentences = [];
    _chapters = [];
    _currentIndex = 0;
    _fetchIndex = 0;
    // 重置分章懒加载模式，防止残留状态触发 _autoAdvanceChapter
    _isDefaultBookMode = false;
    _currentChapterIndex = null;
    _chapterLoadState = ChapterLoadState.idle;
    // 异步清除持久化的当前小说标识
    StorageService.setCurrentNovelId(null)
        .catchError((Object e, StackTrace st) {
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'reader',
        extra: {'context': 'resetForDeletedBook 清除 novelId 持久化失败'},
      );
    });
    notifyListeners();
    CyberLogger.captureMessage(
      '级联重置：书籍 $bookId 已删除，ReaderProvider 已清空',
```

```
      tag: 'reader',
    );
  }
  // ─── 分章懒加载：核心方法 ──────────────────────────────────────────────────────────────────────
  /// 加载指定章节（默认书籍模式专用）
  ///
  /// [resume] 为 true 时从上次进度续读（仅第一次启动时传 true），
  /// 切章/章末推进时传 false（从头开始）。
  Future<void> loadChapter(int chapterIndex, {bool resume = false}) async {
    if (chapterIndex < 0 ||
        chapterIndex >= BookConstants.defaultTotalChapters) {
      return;
    }
    _currentChapterIndex = chapterIndex;
    _isDefaultBookMode = true;
    _chapterLoadState = ChapterLoadState.loading;
    notifyListeners();
    try {
      final service = _defaultBookService ??= DefaultBookService();
      final text = await service.fetchChapter(chapterIndex);
      if (text == null) {
        _chapterLoadState = ChapterLoadState.error;
        notifyListeners();
        return;
      }
      await loadBook(
        text,
        bookId: BookConstants.defaultBookKey,
        chapters: [
          ChapterModel(
            title: BookConstants.xiyoujiChapterTitles[chapterIndex],
            lineIndex: 0,
          ),
        ],
        initialIndex: resume ? null : 0,
      );
      _chapterLoadState = ChapterLoadState.loaded;
      // 持久化章节索引，供热重启恢复
      StorageService.setCurrentChapterIndex(chapterIndex);
      // 影子预读下一章（fire-and-forget）
      service.prefetchNextChapter(chapterIndex);
      notifyListeners();
    } catch (e, st) {
      _chapterLoadState = ChapterLoadState.error;
      notifyListeners();
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'reader',
        extra: {'context': 'loadChapter 失败', 'chapterIndex': '$chapterIndex'},
```

```
      );
    }
  }
  /// 热重启恢复：自动加载上次章节内容
  void restoreDefaultBook() {
    if (_isDefaultBookMode) return;
    final chapterIndex = StorageService.getCurrentChapterIndex();
    CyberLogger.captureMessage('自动恢复默认书第 $chapterIndex 章', tag: 'reader');
    // 先用常量填充标题，让 UI 立即有内容可显示
    _isDefaultBookMode = true;
    _currentChapterIndex = chapterIndex;
    _chapterLoadState = ChapterLoadState.idle;
    if (_chapters.isEmpty) {
      _chapters = BookConstants.xiyoujiChapterTitles
          .asMap()
          .entries
          .map((e) => ChapterModel(title: e.value, lineIndex: e.key))
          .toList();
    }
    // 恢复 _currentIndex，使章节标题显示正确
    _currentIndex = chapterIndex;
    notifyListeners();
    // 异步加载章节正文，完成后提词器自动更新
    loadChapter(chapterIndex, resume: true);
  }
  /// 章末自动推进（TTS 播完当前章节最后一句时由 [onTtsItemFinished] 触发）
  Future<void> _autoAdvanceChapter() async {
    final nextChapter = (_currentChapterIndex ?? -1) + 1;
    if (nextChapter >= BookConstants.defaultTotalChapters) {
      CyberLogger.captureMessage('已到最后一章，停止自动推进', tag: 'reader');
      return;
    }
    CyberLogger.captureMessage('章末自动推进到第 $nextChapter 章', tag: 'reader');
    await loadChapter(nextChapter, resume: false);
  }
  @override
  void dispose() {
    _ttsEngine.removeListener(_onTtsEngineChanged);
    super.dispose();
  }
}
// ===== 文件结束：lib/features/reader/providers/reader_provider.dart =====

// ===== 文件：server/main.go =====
package main
import (
    "context"
    "log"
    "net/http"
    "os"
```

```
0008      "os/signal"
0009      "syscall"
0010      "time"
0012      "github.com/gin-gonic/gin"
0013  )
0015  func main() {
0016      gin.SetMode(gin.ReleaseMode)
0017      r := gin.Default()
0018      if err := r.SetTrustedProxies(nil); err != nil {
0019          log.Fatal(err)
0020      }
0022      if err := initOssBucket(); err != nil {
0023          log.Fatalf("OSS 初始化失败: %v", err)
0024      }
0026      // ——— TTS 语音合成 ——————————————————————————————————————————————————————————————————————
0027      r.POST("/api/v1/tts", ttsHandler)
0029      // ——— 书籍服务 ——————————————————————————————————————————————————————————————————————————
0030      r.GET("/api/v1/book/catalog", bookCatalogHandler)
0031      r.POST("/api/v1/book/chapter", bookChapterHandler)
0033      // ——— 合规页面 ——————————————————————————————————————————————————————————————————————————
0034      r.GET("/privacy", privacyHandler)
0035      r.GET("/ping", func(c *gin.Context) { c.String(200, "pong") })
0037      srv := &http.Server{
0038          Addr:         ":8081",
0039          Handler:      r,
0040          ReadTimeout:  10 * time.Second,
0041          WriteTimeout: 30 * time.Second,
0042          IdleTimeout:  60 * time.Second,
0043      }
0045      go func() {
0046          log.Println("yueyou-server on :8081")
0047          if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {
0048              log.Fatalf("服务启动失败: %v", err)
0049          }
0050      }()
0052      // 优雅关闭：SIGINT / SIGTERM 时等待进行中的请求完成
0053      quit := make(chan os.Signal, 1)
0054      signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
0055      <-quit
0056      log.Println("正在关闭服务...")
0058      ctx, cancel := context.WithTimeout(context.Background(), 15*time.Second)
0059      defer cancel()
0060      if err := srv.Shutdown(ctx); err != nil {
0061          log.Fatalf("服务关闭异常: %v", err)
0062      }
0063      log.Println("服务已退出")
0064  }
// ===== 文件结束：server/main.go =====

// ===== 文件：server/config.go =====
```

```
0001  package main
0003  import "os"
0005  // Config 持有所有运行时配置，优先从环境变量读取，回退到内置默认值。
0006  // 生产部署：通过 systemd EnvironmentFile 或 Docker --env-file 注入。
0007  type Config struct {
0008      AKID        string // 阿里云 AccessKey ID
0009      AKSecret    string // 阿里云 AccessKey Secret
0010      OSSEndpoint string // OSS 内网端点（服务器与 OSS 同 region 时用内网省流量）
0011      OSSBucket   string // OSS Bucket 名称
0012      OSSPub      string // OSS / CDN 公网访问前缀（无尾斜线）
0013  }
0015  // cfg 为全局单例配置，在 main() 启动前由 init() 初始化。
0016  var cfg Config
0018  func init() {
0019      cfg = Config{
0020          AKID:        getenv("YUEYOU_AKID", ""),
0021          AKSecret:    getenv("YUEYOU_AKSEC", ""),
0022          OSSEndpoint: getenv("YUEYOU_OSS_EP", "oss-cn-beijing-internal.aliyuncs.com"),
0023          OSSBucket:   getenv("YUEYOU_OSS_BKT", "general-storage"),
0024          OSSPub:      getenv("YUEYOU_OSS_PUB", "https://general-storage.oss-cn-beijing.aliyuncs.com"),
0025      }
0026  }
0028  func getenv(key, fallback string) string {
0029      if v := os.Getenv(key); v != "" {
0030          return v
0031      }
0032      return fallback
0033  }
// ===== 文件结束：server/config.go =====

// ===== 文件：server/response.go =====
0001  package main
0003  import "github.com/gin-gonic/gin"
0005  // ok 返回成功响应（HTTP 200）。
0006  func ok(c *gin.Context, data gin.H) {
0007      c.JSON(200, data)
0008  }
0010  // fail 返回错误响应，统一 status/error/message 格式。
0011  func fail(c *gin.Context, code int, msg string) {
0012      c.JSON(code, gin.H{"status": "error", "message": msg})
0013  }
// ===== 文件结束：server/response.go =====

// ===== 文件：server/handler_tts.go =====
0001  package main
0003  import (
0004      "bytes"
0005      "crypto/md5"
0006      "fmt"
0007      "log"
```

```
0008      "os"
0009      "os/exec"
0010      "sync"
0012      "github.com/aliyun/aliyun-oss-go-sdk/oss"
0013      "github.com/gin-gonic/gin"
0014  )
0016  // edgeTtsSem 严格串行化 edge-tts 进程，避免触发微软服务端并发任务限制。
0017  var edgeTtsSem = make(chan struct{}, 1)
0019  // inflightMu 保护 inflightKeys，用于对同一文本的并发请求去重。
0020  var (
0021      inflightMu   sync.Mutex
0022      inflightKeys = map[string]chan struct{}{}
0023  )
0025  // allowedVoices 是服务端音色白名单。
0026  // 新增音色时须同步更新客户端 settings_provider.dart 的 loadFromStorage 白名单。
0027  var allowedVoices = map[string]bool{
0028      "zh-CN-XiaoxiaoNeural": true,
0029      "zh-CN-YunxiNeural":    true,
0030      "zh-CN-YunjianNeural":  true,
0031      "zh-CN-XiaoyiNeural":   true,
0032      "zh-CN-XiaomengNeural": true,
0033  }
0035  // maxTextRunes 单次 TTS 请求的最大文本长度（Unicode 字符数）。
0036  const maxTextRunes = 2000
0038  // ttsHandler 接受文本和音色，生成 MP3 并上传 OSS，返回 CDN 播放地址。
0039  // 遵循分离下载原则：客户端 POST 获取 URL，再自行 GET 下载音频。
0040  func ttsHandler(c *gin.Context) {
0041      var req struct {
0042          Text  string `json:"text"  binding:"required"`
0043          Voice string `json:"voice"`
0044      }
0045      if err := c.ShouldBindJSON(&req); err != nil {
0046          fail(c, 400, "JSON error")
0047          return
0048      }
0050      // 输入校验：文本长度上限，防止超大请求耗尽资源
0051      if len([]rune(req.Text)) > maxTextRunes {
0052          fail(c, 400, "文本过长，最多支持 2000 字符")
0053          return
0054      }
0056      // 输入校验：音色白名单
0057      if req.Voice == "" {
0058          req.Voice = "zh-CN-XiaoxiaoNeural"
0059      } else if !allowedVoices[req.Voice] {
0060          fail(c, 400, "不支持的音色")
0061          return
0062      }
0064      hash := md5.Sum([]byte(req.Text + req.Voice))
0065      objKey := fmt.Sprintf("cache/%x.mp3", hash)
0066      objURL := fmt.Sprintf("%s/%s", cfg.OSSPub, objKey)
```

```
0068    if ossBk == nil {
0069        fail(c, 500, "OSS 未初始化")
0070        return
0071    }
0073    // 缓存命中直接返回
0074    if ossExist(objKey) {
0075        ok(c, gin.H{"status": "success", "url": objURL})
0076        return
0077    }
0079    // 同一文本正在合成时，等待已有任务完成后复用 OSS 缓存，避免重复调用 edge-tts
0080    inflightMu.Lock()
0081    if ch, exists := inflightKeys[objKey]; exists {
0082        inflightMu.Unlock()
0083        <-ch
0084        if ossExist(objKey) {
0085            ok(c, gin.H{"status": "success", "url": objURL})
0086        } else {
0087            fail(c, 500, "语音合成失败")
0088        }
0089        return
0090    }
0091    doneCh := make(chan struct{})
0092    inflightKeys[objKey] = doneCh
0093    inflightMu.Unlock()
0094    defer func() {
0095        close(doneCh)
0096        inflightMu.Lock()
0097        delete(inflightKeys, objKey)
0098        inflightMu.Unlock()
0099    }()
0101    log.Printf("[TTS] 合成: %s", req.Text)
0103    // 获取信号量，严格串行化 edge-tts 进程
0104    edgeTtsSem <- struct{}{}
0105    defer func() { <-edgeTtsSem }()
0107    tmpFile := fmt.Sprintf("/tmp/tts_%x.mp3",
0108        md5.Sum([]byte(req.Text+req.Voice+fmt.Sprintf("%d", os.Getpid()))))
0109    defer os.Remove(tmpFile) // 无论成功或 panic，确保临时文件不泄漏
0111    cmd := exec.CommandContext(c.Request.Context(), "edge-tts",
0112        "--text", req.Text,
0113        "--voice", req.Voice,
0114        "--write-media", tmpFile,
0115    )
0116    if out, err := cmd.CombinedOutput(); err != nil {
0117        log.Printf("[TTS] edge-tts 错误: %v, output: %s", err, string(out))
0118        fail(c, 500, "语音合成失败")
0119        return
0120    }
0122    audioData, err := os.ReadFile(tmpFile)
0123    if err != nil {
0124        fail(c, 500, "读取音频失败")
```

```
0125          return
0126      }
0127      if len(audioData) < 100 {
0128          fail(c, 500, "生成音频为空")
0129          return
0130      }
0132      if err := ossBk.PutObject(objKey, bytes.NewReader(audioData)); err != nil {
0133          fail(c, 500, "OSS 上传失败")
0134          return
0135      }
0137      // 上传成功后主动写入缓存，避免下次再发起 IsObjectExist 请求
0138      ossExistCache.Store(objKey, true)
0140      ok(c, gin.H{"status": "success", "url": objURL})
0141  }
0143  var ossBk *oss.Bucket
0144  var ossExistCache sync.Map
0146  func initOssBucket() error {
0147      cli, err := oss.New(cfg.OSSEndpoint, cfg.AKID, cfg.AKSecret)
0148      if err != nil {
0149          return err
0150      }
0151      ossBk, err = cli.Bucket(cfg.OSSBucket)
0152      return err
0153  }
0155  func ossExist(key string) bool {
0156      if _, ok := ossExistCache.Load(key); ok {
0157          // 只有上传成功后才会写入 true，直接信任缓存
0158          return true
0159      }
0160      exists, _ := ossBk.IsObjectExist(key)
0161      if exists {
0162          // 仅缓存"存在"的 key，"不存在"不缓存（防止 stale false-positive）
0163          ossExistCache.Store(key, true)
0164      }
0165      return exists
0166  }
// ===== 文件结束：server/handler_tts.go =====

// ===== 文件：server/handler_book.go =====
0001  package main
0003  import (
0004      _ "embed"
0005      "fmt"
0006      "log"
0008      "github.com/gin-gonic/gin"
0009  )
0011  // xiyoujiCatalogJSON 在编译期嵌入目录文件，换版本时只需替换 data/xiyouji_catalog.json 并重新编译。
0012  //
0013  //go:embed data/xiyouji_catalog.json
0014  var xiyoujiCatalogJSON []byte
```

```
0016 // xiyoujiTotalChapters 总章节数，与客户端 BookConstants.defaultTotalChapters 严格对齐。
0017 const xiyoujiTotalChapters = 100
0019 // bookCatalogHandler 返回西游记目录（章节标题列表）。
0020 //
0021 // GET /api/v1/book/catalog?bookId=xiyouji
0022 //
0023 // 响应格式：{"status":"success","chapters":[{"title":"...","lineIndex":0}...]}
0024 func bookCatalogHandler(c *gin.Context) {
0025     bookID := c.Query("bookId")
0026     if bookID != "xiyouji" {
0027         fail(c, 404, "书籍不存在")
0028         return
0029     }
0030     // 直接返回嵌入的 JSON，客户端按约定解析
0031     c.Data(200, "application/json; charset=utf-8", xiyoujiCatalogJSON)
0032 }
0034 // bookChapterHandler 派发章节 CDN 下载地址，遵循分离下载原则。
0035 //
0036 // POST /api/v1/book/chapter
0037 // 请求体：{"bookId":"xiyouji","chapterIndex":0}
0038 // 响应体：{"status":"success","url":"https://cdn.../books/chapters/xiyouji/001.txt"}
0039 //
0040 // 客户端收到 URL 后自行 GET 下载章节正文，与 TTS 分离下载完全对齐。
0041 func bookChapterHandler(c *gin.Context) {
0042     var req struct {
0043         BookID       string `json:"bookId"       binding:"required"`
0044         ChapterIndex int    `json:"chapterIndex"`
0045     }
0046     if err := c.ShouldBindJSON(&req); err != nil {
0047         fail(c, 400, "参数错误")
0048         return
0049     }
0050     if req.BookID != "xiyouji" {
0051         fail(c, 404, "书籍不存在")
0052         return
0053     }
0054     if req.ChapterIndex < 0 || req.ChapterIndex >= xiyoujiTotalChapters {
0055         fail(c, 400, "章节索引越界")
0056         return
0057     }
0059     // chapterIndex 0-based，OSS 文件名 1-based 三位补零
0060     filename := fmt.Sprintf("%03d.txt", req.ChapterIndex+1)
0061     url := fmt.Sprintf("%s/books/xiyouji/%s", cfg.OSSPub, filename)
0063     log.Printf("[Book] 派发章节: index=%d -> %s", req.ChapterIndex, url)
0065     ok(c, gin.H{
0066         "status": "success",
0067         "url":    url,
0068     })
0069 }
// ===== 文件结束：server/handler_book.go =====
```